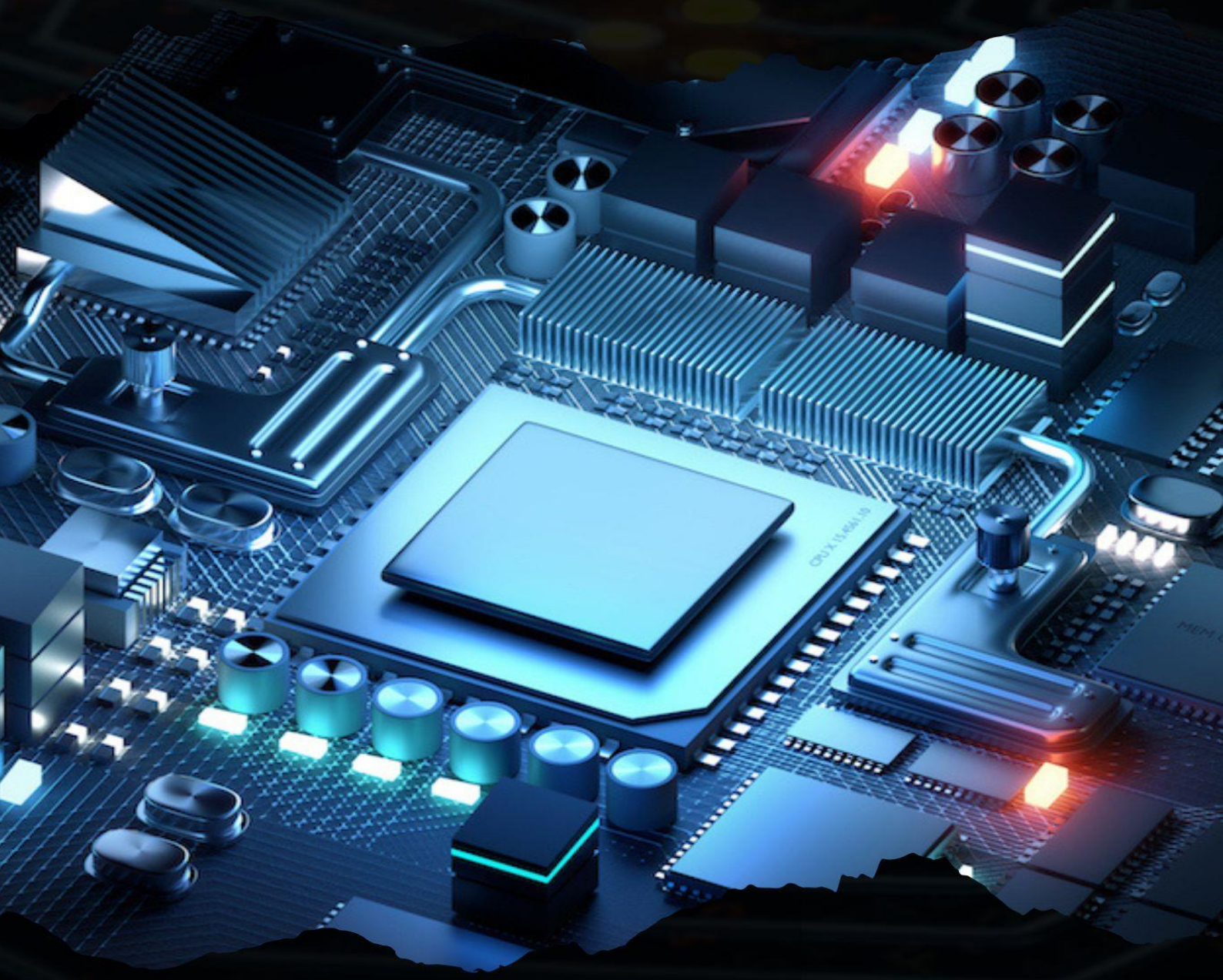
BUKU BESAR

MIKROPROSESOR AND INTERFACING

SEMESTER IV



DISUSUN OLEH
ADISTI, N. ELSADANIKA, M. FAHRIN, N.
2NK - 2023

KETERANGAN

1. Kelompok	: 6	
2. Judul Buku Besar	: Mikroprosesor dan Interfacing Semester IV	
3. Tanggal Dibuat	: 08 Mei 2023	
4. Tanggal Pengumpulan	: 10 Mei 2023	
5. Nama Penulis	: 1) Adisti Nastiti	(211344001)
	2) Elsadanika Maura Sitorus	(211344013)
	3) Fahrin Nisa’	(211344014)
6. Jurusan	: Teknik Elektro	
7. Program Studi	: D4 Teknik Telekomunikasi	
8. Nama Dosen Pembimbing	: 1) Ferry Satria, BSEE., M. T.	
	2) Rahmawati Hasanah., S. ST., M. T.	

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas berkat dan karunia-Nya penulis dapat menyelesaikan buku besar Mikroprosesor dan Interfacing Semester 4 ini tepat pada waktunya. Penulisan buku besar ini diajukan untuk memenuhi salah satu tugas mata kuliah Mikroprosesor and Interfacing.

Dalam penulisan buku besar ini penulis menyampaikan terima kasih kepada:

1. Orang tua dan keluarga yang senantiasa memberian semangat serta dukungan moral maupun material.
2. Bapak Ferry Satria, BSEE., M.T. dan Ibu Rahmawati Hasanah, S. ST., M. T. selaku pembimbing yang telah membantu penulis dalam penyusunan karya tulis ilmiah ini.
3. Teman-teman kelas Nirkabel Angkatan 2021 yang senantiasa memberi dukungan moral.
4. Serta pihak lain yang tidak dapat penulis sebutkan satu persatu atas bantuan secara langsung maupun tidak langsung sehingga karya tulis ilmiah ini dapat terselesaikan dengan baik.

Kami menyadari bahwa buku besar ini masih jauh dari sempurna, oleh karena itu penulis mengharapkan kritik dan saran yang bersifat membangun. Semoga buku besar ini bermanfaat bagi kalangan Politeknik Negeri Bandung bahkan masyarakat luas.

Bandung, Mei 2023

Penulis

DAFTAR ISI

KETERANGAN 2
KATA PENGANTAR 3
DAFTAR ISI 4
BAB I MIKROPROSESOR 6
1.1 Sejarah 6
1.2 Perkembangan Mikroprosesor 7
1.3 Sistem Mikroprosesor 13
1.4 Register, Penyimpanan dan Perintah Jump 15
1.4.1 Register 15
1.4.2 Penyimpanan 19
1.4.3 Perintah Jump 20
1.5 Addressing Mode 21
1.5.1 Pengertian dan Ciri Addressing Mode 21
1.5.2 OP Code 23
1.6 Instruksi Interrupt 25
1.6.1 Pengertian Interrupt 25
1.6.2 Perintah - perintah Interrupt 26
1.7 Overflow 27
BAB II PERINTAH PENJUMLAHAN 29
2.1 Penjumlahan Data Desimal 29
2.2 Penjumlahan Bilangan Oktal 30
BAB III KONVERSI DATA 32
3.1 Desimal ke Heksadesimal 32
3.2 Heksadesimal ke Desimal 33
3.3 Oktal ke Heksadesimal 33
3.4 Heksadesimal ke Octal 33
3.5 Biner ke Gray 34
BAB IV PERINTAH PERKALIAN DAN PEMBAGIAN 35
4.1 Perkalian Heksadesimal 35
4.1.1 Metode Penjumlahan Berulang 35
4.1.2 Metode Shift and Add 1 35
4.1.3 Metode Shift and Add 2 36
4.1.4 Metode MUL 37

4.2 Pembagian Heksadesimal 38
4.2.1 Metode Pengurangan Berulang 38
4.2.2 Metode Shift and Subtract 38
4.2.3 Metode DIV 38
4.2.4 Metode Komplemen 10 38
BAB V PERINTAH MENGGAMBAR 40
BAB VI PROGRAM KALKULATOR SEDERHANA 42
BAB VII LOOK-UP TABLE METHOD 47
BAB VIII INTERFACING I/O DAN MEMORY 50
8.1 Interfacing I/O 50
8.1.1 I/O Instructions 50
8.1.2 Isolated dan Memory-Mapping I/O 50
8.1.3 Basic Input Interface 52
8.1.4 Basic Output Interface 52
8.1.5 Handshaking 53
8.1.6 Input Device 54
8.1.7 Output Device 55
8.1.8 I/O Port Addressing Decoding 56
8.1.9 8 and 16-bit Wide I/O Ports 57
8.1.10 Programmable Peripheral Interface 59
8.1.11 Basic Description of the 82C55 60
8.1.12 Programming The 82C55 61
8.1.13 Programming the 8254 65
8.1.14 Reading a Counter 66
8.1.15 Analog-To-Digital (ADC) and Digital-To-Analog (DAC) Converters 72
8.2 Interfacing Memory 76
8.3 LATIHAN 80
DAFTAR PUSTAKA 85

BAB I
MIKROPROSESOR

1.1 Sejarah

Mikroprosesor adalah sebuah IC (Integrated Circuit) yang digunakan sebagai otak/pengolah utama dalam sebuah sistem komputer. Mikroprosesor merupakan hasil dari pertumbuhan semikonduktor. Prosesor adalah chip yang sering disebut "Microprosessor" yang sekarang ukurannya sudah mencapai gigahertz. Ukuran tersebut adalah hitungan kecepatan prosesor dalam mengolah data atau informasi. Merk prosesor yang banyak beredar dipasatan adalah AMD, Apple, Cyrix VIA, IBM, IDT, dan Intel. Pertama kali Mikroprosesor dikenalkan pada tahun 1971 oleh Intel Corp, yaitu Mikroprosesor Intel 4004 yang mempunyai arsitektur 4 bit. Dengan penambahan beberapa peripheral (memori, piranti I/O, dsb) Mikroprosesor 4004 di ubah menjadi komputer kecil oleh intel. Kemudian mikroprosesor ini di kembangkan lagi menjadi 8080 (berasitektur 8bit), 8085, dan kemudian 8086 (berasitektur 16bit). Microprocessor 4004 Intel meluncurkan mikroprosesor pertama di dunia, 4-bit 4004, yang didesain oleh Federico Faggin. Microprocessor 4004.Processor di awali pada tahun 1971 dimana intel mengeluarkan processor pertamanya yang di pakai pada mesin penghitung buscom. Ini adalah penemuan yang memulai memasukan system cerdas kedalam mesin.

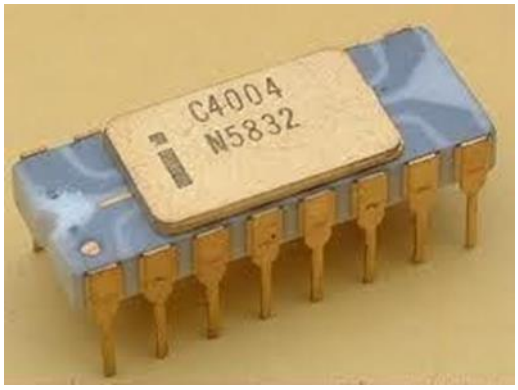
Mikroprosesor 4004 mempunyai 2.250 transistor PMOS, menangani data 4 bit, dan dapat mengeksekusi 60 ribu operasi per detik. Mikroprosesor 4004 ini adalah salah satu dari seri IC untuk komponen kalkulator tersebut: 4001: memori ROM 2.048 bit; 4002: memori RAM MIKROPROSESOR 8008 Pada tahun 1972 intel mengeluarkan microprocessor 8008 yang berkecepatan hitung 2 kali lipat dari MP sebelumnya. MP ini adalah Mikroprosesor 8 bit pertama. Mp ini juga di desain untuk mengerjakan satu pekerjaan saja. Bill Gates muda dan Paul Allen coba mengembangkan bahasa pemograman untuk chip tersebut, namun saat itu masih kurang kuat.

Mikroprosesor 8080 Pada tahun 1974 intel kembali mengeluarkan Mikroprosesor terbaru dengan seri 8080, dengan 4.500 transistor yang memiliki kinerja 10 kali pendahulunya. Pada seri ini intel melakukan perubahan dari Mikroprosesor multivoltage menjadi triple voltage, teknologi yang di pakai NMOS, lebih cepat dari seri sebelumnya yang memakai teknologi PMOS. Mikroprosesor ini adalah otak

pertama bagi komputer yang bernama altair. Pada saat ini pengalamatan memory sudah sampai 64 kilobyte. Kecepatanya sampai 10X mp sebelumnya.

1.2 Perkembangan Mikroprosesor

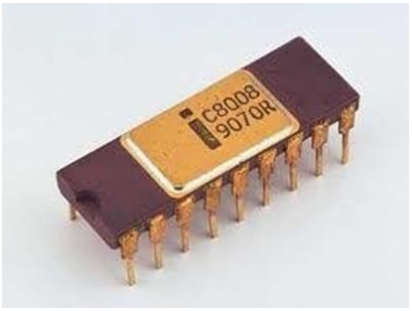
Mikroprosesor pertama kali diperkenalkan oleh Intel Corporation pada tahun 1971 berupa sebuah chip Intel 4004. Chip 4004 berupa LSI (Large Scale Integration) yang berisi rangkaian logika yang cukup besar, karena chip LSI mampu menggantikan ratusan rangkaian yang digunakan dalam sistem mikro komputer konvensional yang digunakan pada waktu itu. Pada 4004 memiliki 46 perintah (instruction) dan mampu memuat data sebanyak 4 bit setiap saat. Selanjutnya mikroprosesor tersebut dikembangkan menjadi 8 bit data dan jumlah instruksi diperbanyak menjadi 48 instruksi dan nama barunya adalah 8008. Pada chip 8008 memiliki kecepatan proses yang lebih tinggi daripada 4004.



Gambar 1.1 Mikroprosesor intel 4004

Chip 4004 dan 8008 digunakan secara luas dalam berbagai bidang dan mampu memacu perkembangan industri dalam waktu yang sangat singkat dan hanya dalam waktu 2 tahun berikutnya telah dikembangkan mikroprosesor standar 8 bit tipe 8080 yang merupakan penyempurnaan dari 8008 sehingga lebih canggih dan memiliki kecepatan yang lebih tinggi serta jumlah instruksi yang lebih banyak. Dan satu tahun kemudian pabrik-pabrik komponen elektronik yang lain terinspirasi dengan intel 8080 dengan mengeluarkan berbagai macam mikroprosesor 8 bit dengan tipe yang sesuai dengan pabrik pembuatnya. Misalnya Motorola dengan tipe 6800, Signetic dengan tipe 2650, Rokwell dengan PPS-8 dan sebagainya. Pada tahun 1972, 8008 dengan bus data 8 bit digunakan oleh Don Lancaster untuk membuat cikal-bakal

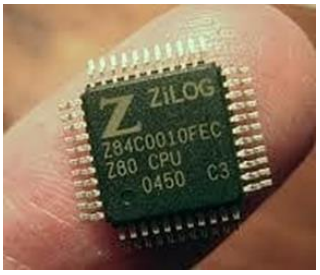
personal komputer. 8008 membutuhkan 20 komponen tambahan untuk dapat bekerja penuh sebagai CPU.



Gambar 1.2 Mikroprosesor intel 8008

Tahun 1974, 8080 menjadi otak personal pertama komputer, Altair, diduga merupakan nama tujuan pesawat Starship Enterprise di film TV Star Trek. 8080 hanya membutuhkan 2 perangkat tambahan untuk bekerja. Selain itu 8080 terbuat dari transistor NMOS yang bekerja lebih cepat. 8080 disebut sebagai mikroprosesor generasi kedua. Segera sesudah itu Motorolla membuat MC6800 yang juga merupakan CPU multiguna. MC6800 sangat populer karena menggunakan catu daya +5V, dibanding 8080 dengan catu daya –5V, +5V, -12V, dan +12V.

Pada tahun 1975 muncul mikroprosesor generasi ketiga yang dikeluarkan oleh Zilog Inc, yaitu dengan tipe Z80 yang mampu menutup kekurangan dari intel 8080 baik dari segi kecepatan, jumlah instruksi dan kemudahan dalam operasionalnya, dan pada saat itu pula pabrik-pabrik yang lain mulai berlomba-lomba untuk saling mengungguli antara yang satu dengan yang lain. Bahkan dikembangkan pula komputer dalam satu chip (single chip microcomputer) misalnya F8 dari Fairchild dan Mostek 8048 dari Intel dan lain-lainnya.



Gambar 1.3 Mikroprosesor Zilog 80 dari Motorola

Tahun 1978, IBM menciptakan personal komputer PC-XT yang sangat populer menggunakan mikroprosesor 8086 dan 8088. Keduanya mampu menangani data 16 bit. Bedanya hanya pada ukuran bus data yang hanya 8 bit untuk 8088 (operasi

internal 16 bit), dan 16 bit untuk 8086. Kemudian Intel membut 80186 dan 80188 yang juga berisi perangkat peripheral terprogram. Tahun 1982, 80286 adalah prosesor pertama yang dapat menjalankan perangkat lunak yang ditulis untuk pendahulunya, karena instruksi yang dimiliki oleh seri sebelumnya semuanya dimiliki dan ditambahi dengan instruksi lain. Kompatibilitas ke atas ini kemudian menjadi ciri khas mikroprosesor Intel. Dalam 6 tahun, ada 15 juta PC-AT yang menngunakan 80286 sebagai CPU.

Tahun 1985, Intel membuat 80386 (386TM) yang mengandung 275 ribu transistor, dan merupakan mikroprosesor 32 bit yang dapat melakukan multi tasking (menjalankan beberapa program dalam waktu yang bersamaan). Tahun 1989, Intel 486TM adalah prosesor pertama yang mempunyai math coprosesor secara built-in di dalamnya.



Gambar 1.4 Mikroprosesor intel 80386

Tahun 1993, lahir keluarga prosesor Pentium®. Tahun 1995, prosesor Pentium® Pro didesain untuk server 32-bit, mengandung 5,5 juta transistor dan mempunyai chip memori cache kedua di dalamnya. Tahun 1997, dibuat prosesor Pentium® II dengan 7,5 juta transistor dan teknologi MMX, yang didesain khusus untuk memproses data video, audio and grafik secara efisien. Prosesor ini juga diperkenalkan dengan bentuk cartridge Single Edge Contact (S.E.C). Seiring dengan itu bermunculan seri Celeron yang merupakan versi Pentium dengan beberapa fitur yang dihilangkan untuk menekan biaya produksi.

Tahun 1999 muncul Pentium III dengan 70 instruksi baru yang mendukung Internet Streaming SIMD. Prosesor ini berisi 9,5 juta transistor, dan mengintroduksi teknologi 0,25-micron. Pada saat ini sedang dikembangkan mikroprosesor 64 bit, sehingga operasi-operasi matematis yang dilakukan dapat lebih cepat. Prosesor Intel Celeron merupakan prosesor yang dikeluarkan sebagai prosesor yang ditujukan untuk

pengguna yang tidak terlalu membutuhkan kinerja prosesor yang lebih cepat bagi pengguna yang ingin membangun sebuah system computer dengan budget (harga) yang tidak terlalu besar. Prosesor Intel Celeron ini memiliki bentuk dan form factor yang sama dengan prosesor Intel jenis Pentium, tetapi hanya dengan instruksi-instruksi yang lebih sedikit, L2 cache-nya lebih kecil, kecepatan (clock speed) yang lebih lambat, dan harga yang lebih murah daripada prosesor Intel jenis Pentium. Dengan keluarnya prosesor Celeron ini maka Intel kembali memberikan sebuah prosesor untuk sebuah pasaran tertentu.



Gambar 1.5 Bentuk Intel Pentium III Internet Streaming SIMD

Tahun 2000 Intel mengeluarkan produk terbarunya yaitu Intel® Pentium® 4 Prosesor, prosesor Pentium 4 merupakan produk Intel yang kecepatan prosesnya mampu menembus kecepatan hingga 3.06 GHz. Pertama kali keluar prosesor ini berkecepatan 1.5GHz dengan formafactor pin 423, setelah itu intel merubah formfactor prosesor Intel Pentium 4 menjadi pin 478 yang dimulai dari prosesor Intel Pentium 4 berkecepatan 1.3 GHz sampai yang terbaru yang saat ini mampu menembus kecepatannya hingga 3.4 GHz.

Tahun 2001 intel memperbarui prosessornya yaitu Intel® Xeon® Prosesor, prosesor Intel Pentium 4 Xeon merupakan prosesor Intel Pentium 4 yang ditujukan khusus untuk berperan sebagai computer server. Prosesor ini memiliki jumlah pin lebih banyak dari prosesor Intel Pentium 4 serta dengan memori L2 cache yang lebih besar pula. Pada tahun yang sama intel juga mengeluarkan produk yang lain yang dinamakan Itanium. Itanium adalah prosesor pertama berbasis 64 bit yang ditujukan bagi pemakain pada server dan workstation serta pemakai tertentu. Prosesor ini sudah dibuat dengan struktur yang benar-benar berbeda dari sebelumnya yang didasarkan pada desain dan teknologi Intel's Explicitly Parallel Instruction Computing (EPIC).



Gambar 1.6 Layout mikroprosesor intel pentium 4

Tahun 2002 Intel memperbaiki produk Itaniumnya yaitu Intel® Itanium® 2 Prosesor. Itanium 2 adalah generasi kedua dari keluarga Itanium.

Tahun 2003 intel mengeluarkan produk barunya yaitu Intel® Pentium® M Prosesor, dimana Chipset 855, dan Intel® PRO/WIRELESS 2100 adalah komponen dari Intel® Centrino™. Intel Centrino dibuat untuk memenuhi kebutuhan pasar akan keberadaan sebuah komputer yang mudah dibawa kemana-mana.

Tahun 2004 mengeluarkan produk Intel Pentium M 735/745/755 prosesors dan Intel E7520/E7320 Chipsets dimana dilengkapi dengan chipset 855 dengan fitur baru 2Mb L2 Cache 400MHz system bus dan kecocokan dengan soket prosesor dengan seri-seri Pentium M sebelumnya, sedangkan 7320/7520 dapat digunakan untuk dual prosesor dengan konfigurasi 800MHz FSB, DDR2 400 memori, and PCI Express peripheral interfaces. Intel memperbaiki produknya pada tahun 2005 dengan mengeluarkan Intel Pentium 4 Extreme Edition 3.73GHz, dimana prosesor yang ditujukan untuk pasar pengguna komputer yang menginginkan sesuatu yang lebih dari komputernya, prosesor ini menggunakan konfigurasi 3.73GHz frequency, 1.066GHz FSB, EM64T, 2MB L2 cache, dan Hyper Threading.



Gambar 1.7 Intel pentium 4 extreme edition 3.73GHz

Pada tahun yang sama mengeluarkan juga Intel Pentium D 820/830/840 yaitu Prosesor berbasis 64 bit dan disebut dual core karena menggunakan 2 buah inti, dengan konfigurasi 1MB L2 cache pada tiap core, 800MHz FSB, dan bisa beroperasi

pada frekuensi 2.8GHz, 3.0GHz, dan 3.2GHz. Pada prosesor jenis ini juga disertakan dukungan Hyper Threading.

Tahun 2006 mengeluarkan lagi produk Intel Core 2 Quad Q6600 dimana Prosesor untuk type desktop dan digunakan pada orang yang ingin kekuatan lebih dari komputer yang ia miliki memiliki 2 buah core dengan konfigurasi 2.4GHz dengan 8MB L2 cache (sampai dengan 4MB yang dapat diakses tiap core), 1.06GHz Front-side bus, dan thermal design power (TDP). Tahun 2006 juga mengeluarkan produk Intel Quad-core Xeon X3210/X3220 yaitu Prosesor yang digunakan untuk tipe server dan memiliki 2 buah core dengan masing-masing memiliki konfigurasi 2.13 dan 2.4GHz, berturut-turut, dengan 8MB L2 cache (dapat mencapai 4MB yang diakses untuk tiap core), 1.06GHz Front-side bus, dan thermal design power (TDP).

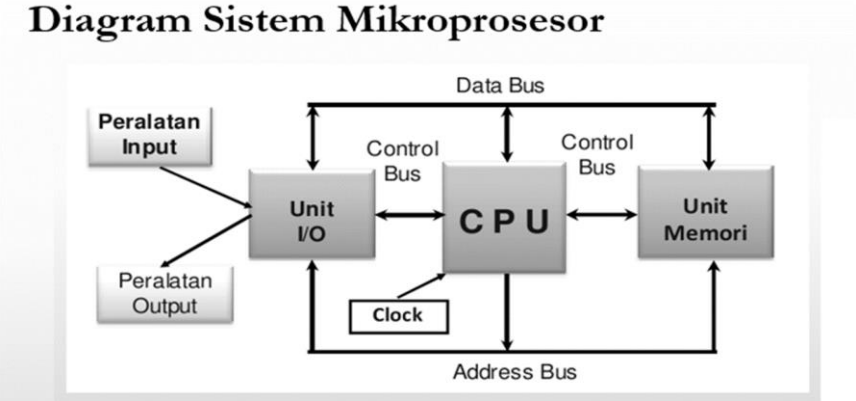
Tahun 2007 Intel memperkenalkan prosesor core 2 quad di bulan januari. Processor untuk type desktop dan digunakan pada orang yang ingin kekuatan lebih dari komputer yang ia miliki memiliki 2 buah core dengan konfigurasi 2.4GHz dengan 8MB L2 cache (sampai dengan 4MB yang dapat diakses tiap core), 1.06GHz Front-side bus, dan thermal design power (TDP).

Intel Core i3 diluncurkan dengan dua tipe, yaitu untuk desktop processor dan mobile processor (notebook). Untuk tipe desktop, Intel Core i3 menggunakan microarchitecture yang diberi codename Clarkdale, yang memiliki L3 Cache sebesar 4 MiB, dengan Thermal Design Power (TDP) sebesar 74 Watt.

Intel Core i5 yang memiliki kelebihan yaitu ditanamkannya fungsi chipset Northbridge pada inti processor (dikenal dengan nama MCH pada Motherboard). Maka motherboard Core i5 yang akan menggunakan chipset Intel P55 (dikelas mainstream) ini akan terlihat lowong tanpa kehadiran chipset northbridge.

Intel Core i7 adalah Processor pertama dengan teknologi “Nehalem”. Nehalem menggunakan platform baru yang betul-betul berbeda dengan generasi sebelumnya. Salah satunya adalah mengintegrasikan chipset MCH langsung di processor, bukan motherboard. Nehalem juga mengganti fungsi FSB menjadi QPI (Quick Path Interconnect) yang lebih revolusioner.

1.3 Sistem Mikroprosesor



Gambar diatas menunjukkan komponen-komponen utama dari suatu sistem komputer berbasis mikroprosesor, antara lain:

- CPU (Central Processing Unit) = Komponen utama pada sistem mikroprosesor, unit ini bekerja sebagai pusat pengolah data atau pengendali sistem.
- Unit Penyimpanan Memori [RAM (Random Access Memory), ROM(Read Only Memory)] = Untuk merekam data/program. Unit memory mempunyai alamat untuk menyimpan data/program.
- Perangkat Input/Output (I/O) = Tempat masuk/keluarnya data ke/dari sistem. Port memiliki alamat.

CPU adalah tempat dilakukannya operasi aritmatika dan logika. CPU berisi register-register, clock frekuensi tinggi, Control Unit (CU), unit aritmatika logika dan Arithmetic Logic Unit (ALU). Berikut penjelasan setiap komponen:

a) Clock

   Clock berfungsi untuk mensinkronisasi operasi internal CPU dengan komponen

   sistem lainnya menggunakan pulsa clock pada rate/kecepatan yang konstan (unit dasar waktu untuk instruksi mesin adalah machine cycle atau clock cycle). Sebuah instruksi mesin membutuhkan setidaknya satu clock cycle.

b) Control Unit (CU)

   CU berfungsi membangkitkan sinyal kontrol yang dibutuhkan untuk mengkoordinasikan urutan langkah-langkah yang terlibat dalam

pengeksekusian instruksi mesin. Langkah-langkah tersebut antara lain; mengambil data daninstruksi, dan mendekodekan alamat untuk ALU.

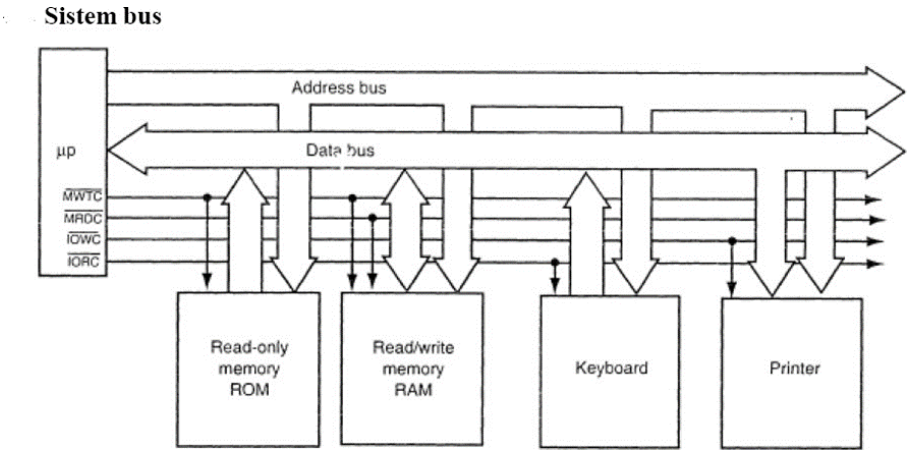
c) Arithmetic Logic Unit (ALU)

ALU melakukan operasi aritmatika seperti penambahan dan pengurangan, dan operasi logika seperti AND,OR dan NOT.

d) Register

Register merupakan lokasi memori internal.

e) Sistem Bus



Komponen-komponen dalam sistem komputer berkomunikasi dengan komponen lainnya dan juga dengan dunia di luar sistem menggunakan sebuah bus. Bus adalah sekumpulan kabel yang mengirimkan data dari satu bagian komputer ke bagian lainnya. Ada beberapa tipe bus, yaitu:

1. *Address Bus*: menentukan jumlah dari memori fisik yang dapat dialamatkan oleh mikroprosesor dan itu menentukan alamat instruksi dan data, ketika instruksi yang sedang dijalankan mengirimkan data antara CPU dan memori.
2. *Data Bus*: mengirimkan instruksi dan data, serta mengindikasikan ukuran data yang dikirimkan antara mikroprosesor dan memori atau perangkat I/O.
3. *Control Bus*: *control bus* menyinkronisasikan setiap perangkat yang terhubung ke system bus, menggunakan sinyal biner. Control Bus terdiri dari sinyal kontrol seperti *memory read*, *memory write*, *I/O read*, *I/O write*, *interrupt acknowledge*.

CPU juga memiliki fungsi-fungsi dasar yaitu:

1) Dapat mengenali lokasi memory tempat instruksi atau data berada.

2) Melakukan proses pengambilan instruksi atau data.
3) Dapat menyimpan sementara instruksi atau data sampai instruksi dan data tersebut dieksekusi.
4) Mengenali, mengerti, dan dapat menerjemahkan setiap instruksi.
5) Dapat mengeksekusi instruksi.
6) Dapat mengkoordinasikan semua proses sehingga dilakukan dalam urutan langkah yang benar.
7) Mengulangi semua urutan proses selama masih ada instruksi yang harus dieksekusi.

1.4 Register, Penyimpanan dan Perintah Jump

1.4.1 Register

Mikroprosesor 8086 mempunyai total 14 register yang dapat diakses oleh programmer. Seorang programmer perlu mengetahui berbagai macam register yang terdapat di EU dan BIU. Register-register tersebut antara lain:

a) Register General Purpose

Register General Purpose adalah register 16 bit yang terdiri dari register AX, BX, CX, dan DX. Fungsi dari masing-masing register ini ialah:

- AX: Register AX dapat berfungsi secara terpisah sepanjang 8 bit yaitu bagian high byte (AH) dan bagian low byte (AL). AX merupakan register akumulator yang biasa dipakai untuk proses aritmatika perkalian dan pembagian, proses logika, dan pengiriman data. AX merupakan register yang membangkitkan kode mesin tercepat.
- BX: Register BX dapat berfungsi secara terpisah sepanjang 8 bit yaitu bagian high byte (BH) dan bagian low byte (BL). BX merupakan register alamat dasar.
- CX: Register CX dapat berfungsi secara terpisah sepanjang 8 bit yaitu bagian high byte (CH) dan bagian low byte (CL). Register CX merupakan register yang biasa dijadikan counter (penghitung). CX menghitung code segment secara berulang menggunakan instruksi loop. CX juga digunakan untuk mengulang-ulang suatu operasi pada string menggunakan perintah REP. Selain itu, register CL juga

digunakan sebagai tempat untuk menyimpan data seberapa banyak bit pergeseran yang diinginkan pada perintah shift dan rotate.

- DX: Register DX dapat berfungsi secara terpisah sepanjang 8 bit yaitu bagian high byte (DH) dan bagian low byte (DL). Register DX merupakan register data. Penggabungan DX:AX menjadi 32-bit dijadikan sebagai tempat penyimpanan data oleh operasi MUL dan DIV. Register DX juga berfungsi untuk menentukan port dalam beberapa operasi IN dan OUT.

b) Register IP (Instruction Pointer)

Register IP yaitu adalah register khusus dalam sebuah prosesor. Register IP berisi alamat instruksi berikutnya yang akan dieksekusi.. Terdapat empat register yang termasuk Register IP, yaitu:

- SI (Source Index): register SI dapat digunakan untuk pengalamatan pointer data, sebagai sumber dalam beberapa intruksi pemrosesan string, dan sebagai alamat offset relative terhadap DS.
- DI (Destination Index): register DI dapat digunakan untuk pengalamatan pointer data, sebagai tujuan dalam beberapa intruksi pemrosesan string, dan sebagai alamat offset relative terhadap ES.
- BP (Base Pointer): register SI digunakan untuk mengakses parameter yang dilewatkan melalui stack, dan sebagai alamat offset relative terhadap SS.
- SP (Stack Pointer): register SP selalu menunjukkan item teratas pada stack. Stack yang kosong akan memiliki SP=FFFEh. Register SP digunakan sebagai alamat offset relative terhadap SS.

c) Register Segment

Register Segment merupakan register tambahan dengan panjang 16 bit atau terbatas sampai 64K byte, yang berfungsi men-generate alamat memori ketika dikombinasikan dengan register lain. Register segment berfungsi untuk menyimpan alamat segment, yang terdiri dari CS (Code Segment), DS (Data Segment), SS (Stack Segment), dan ES (Extra Segment).

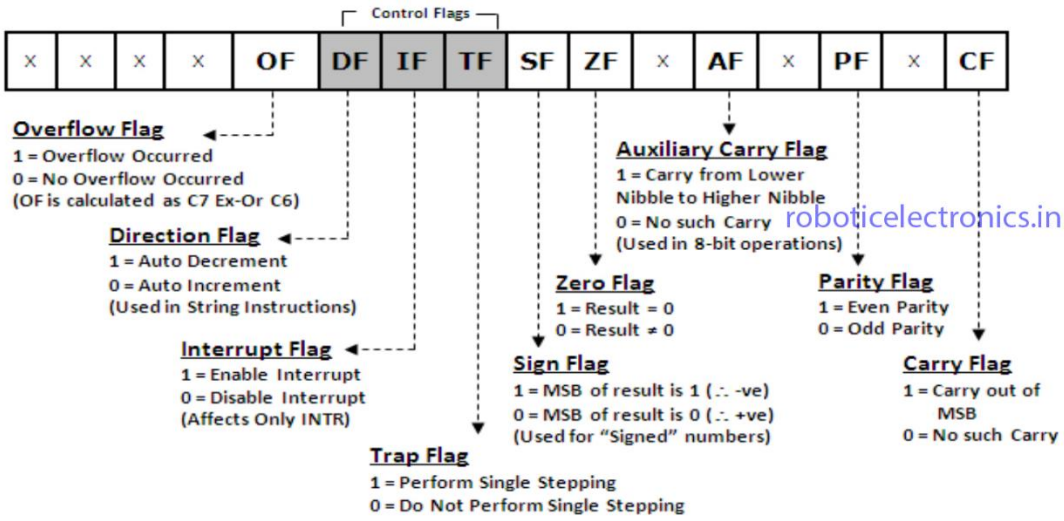
- Register CS (Code Segment), bagian memori 64K-byte yang digunakan untuk menyimpan alamat segment dari kode program atau

instruksi. Register ini sering berubah oleh suatu instruksi jump, panggilan, atau return.

- Register DS (Data Segment), bagian memori 64K-byte yang berisi data yang menjadi referensi untuk hampir semua instruksi dan sebagian besar mode addressing. Register DS ini digunakan untuk menyimpan alamat segment dari letak data. Data hampir selalu bergerak masuk atau keluar dari memori melalui segment data.
- Register SS (Stack Segment), bagian memori 65K-byte yang digunakan untuk menyimpan alamat segment memori yang dipergunakan menjadi Stack (tumpukan). Address Stack yang efektif ialah suatu kombinasi isi dari : Stack Pointer (SP) + SS x 10H. Misal SS berisi 1000H dan SP berisi 0000H. Maka addres stack ditempatkan di 0000+10000 = 10000H. Dapat juga ditulis 10000:0000 ; address segment dari 1000H dan addres offset dari 0000H.
- Register ES (Extra Segment), atau segmen tambahan, merupakan register khusus digunakan untuk instruksi string dan meyimpan alamat segment tambahan, misalnya alamat display, alamat sistem operasi, dan lain-lain.

d) Flag Register

Operasi-operasi atau instruksi yang dijalankan mikroprosesor dapat menghasilkan status flag tertentu. Dari kondisi flag ini kita akan mengetahui kondisi dari suatu proses yang ada di dalam program, sehingga memudahkan kita untuk mengambil keputusan atau untuk mengatur program selanjutnya. Misal, kita memiliki suatu counter yang berisi data banyaknya pengulangan suatu program. Setiap kali program selesai dikerjakan, counter akan berkurang satu dan kemudian kembali lagi menjalankan program hingga counter bernilai nol baru program akan berhenti. Untuk tahu counter telah bernilai nol, kita dapat mengetahuinya dari status flag. Flag register atau disebut juga status register adalah suatu register 16 bit yang berisi flag sebagai berikut:



- CF (Carry Flag): Flag ini mengindikasikan adanya carry hasil suatu penjumlahan atau borrow dari suatu pengurangan. Jika CF bernilai 1 berarti terdapat carry atau borrow selain itu CF=0. Fungsi lainnya, CF menyimpan juga digunakan sebagai indikator perkalian dan indicator kondisi overflow untuk aritmatika bilangan integer tak-bertanda.
- PF (Parity Flag): Flag ini menunjukkan nilai paritas suatu hasil perhitungan atau proses logika. Jika hasil dari proses tersebut memiliki cacah genap dari logika 1 maka PF=1, selain itu PF bernilai nol. Sebagai contoh, jika hasil suatu operasi adalah 10101010, cacah logika 1 nya =4 maka PF=1. Jika hasil suatu operasi adalah 10101000, cacah logika 1 nya =3 maka PF=0.
- AF (Auxiliary Carry Flag): berfungsi sama seperti CF, namun AF mengindikasikan adanya carry hasil suatu penjumlahan atau borrow dari

  suatu pengurangan antara setengah byte (dari lower nibble misal D0-D3

  ke upper nibble D4-D7) dengan menggunakan register AL. AF bernilai 1 jika terdapat carry yang diberikan dari D3 ke D4. Flag ini bukanlah flag yang digunakan untuk tujuan umum, flag ini digunakan secara internal oleh prosesor untuk menjalankan konversi Binary-to-BCD, dimana BCD merupakan sistem bilangan desimal yang dikodekan secara biner, hanya mengenal angka 0-9 dalam biner.

- ZF (Zero Flag): Flag ini mengindikasikan hasil suatu operasi aritmatika
  atau logika bernilai 0. Jika suatu operasi aritmatika atau logika bernilai 0 maka ZF=1, selain itu ZF=0.
- SF (Sign Flag): Flag ini beroperasi untuk bilangan bertanda. Jika hasil suatu operasi (aritmatika, logika, pergeseran atau rotasi) merupakan bilangan negatif, maka SF bernilai 1.
- OF (Overflow Flag): Flag ini mengindikasikan adanya kesalahan selama
  operasi bilangan bertanda. OF bernilai 1 jika terdapat hasil operasi yang
  melampaui kapasitas mesin.
- IF (Interrupt-enable flag). Flag ini mengindikasikan dapat atau tidak dapat sebuah instruksi melakukan interupsi.
- DF (Direction Flag). Flag ini dipergunakan untuk mengontrol arah dari
  operasi string.
- TF (Trap Flag). Flag ini ditempatkan dalam mode single step untuk keperluan debug.

1.4.2 Penyimpanan

Assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte. Nama variabel digunakan untuk ruang penyimpanan yang terkait dengan nilai offset dalam data segmen. Berikut adalah bentuk-bentuk dasar dari directive define:

Perintah	Keterangan	Storage Space
DB	Define Byte	1 byte
DW	Define Word	2 bytes
DD	Define Doubleword	4 bytes
DQ	Define Quadword	8 bytes
DT	Define Ten Bytes	

1.4.3 Perintah Jump

Perintah percabangan ada 2 tipe yaitu perintah pada bilangan tidak bertanda dan perintah jump pada bilangan bertanda. Berikut penjelasannya:

- Perintah Jump pada bilangan tidak bertanda:
  a) JA (Jump If Above) = Melakukan suatu loncatan menuju "tujuan" bila CF=0 dan ZF=0.
  b) JNBE (Jump If Not Bellow or Equal) = Melakukan suatu loncatan menuju "tujuan" bila CF=0 dan ZF=0.
  c) JAE (Jump If Above or Equal) = Melakukan suatu loncatan menuju "tujuan" bila CF=0.
  d) JNB (Jump If Not Bellow) = Melakukan suatu loncatan menuju "tujuan" bila CF=0.
  e) JB (Jump If Bellow) = Melakukan suatu loncatan menuju "tujuan" bila CF=1.
  f) JNAE (Jump If Not Above or Equal) = Melakukan suatu loncatan menuju "tujuan" bila CF=1.
  g) JBE (Jump If Below or Equal) = Melakukan suatu loncatan menuju "tujuan" bila CF=1 atau ZF=1.
  h) JNA (Jump If Not Above) = Melakukan suatu loncatan menuju "tujuan" bila CF=1 atau ZF=1.
  i) JC (Jump On Carry) = Memiliki fungsi yang sama dengan JB dan JNAE.
  j) JNP (Jump On No Parity) = Melakukan suatu loncatan menuju "tujuan" bila PF=0.
  k) JPO (Jump On Parity Odd) = Melakukan suatu loncatan menuju "tujuan" bila PF=0.
- Perintah Jump pada bilangan bertanda:
  a) JG (Jump If Greater) = Melakukan suatu loncatan menuju "tujuan" bila ZF=0 dan SF=OF.
  b) JNLE (Jump If Not Less or Equal) = Melakukan suatu loncatan menuju "tujuan" bila ZF=0 dan SF=OF.
  c) JGE (Jump If Greater or Equal) = Melakukan suatu loncatan menuju "tujuan" bila SF=OF.
  d) JNL = Melakukan suatu loncatan menuju "tujuan" bila SF=OF.

e) JL (Jump If Less Than) = Melakukan suatu loncatan menuju "tujuan" bila SF tidak sama dengan OF.
f) JNGE (Jump If Not Greater or Equal) = Melakukan suatu loncatan menuju "tujuan" bila SF tidak sama dengan OF.
g) JLE (Jump If Less Than or Equal) = Melakukan suatu loncatan menuju "tujuan" bila ZF=1 atau SF tidak sama dengan OF.
h) JNG (Jump If Not Greater) = Melakukan suatu loncatan menuju "tujuan" bila ZF=1 atau SF tidak sama dengan O
i) JNO (Jump On Not Overflow) = Melakukan suatu loncatan menuju "tujuan" bila OF=0.
j) JNS (Jump On No Sign) = Melakukan suatu loncatan menuju "tujuan" bila SF=0.
k) JO (Jump On Overflow) = Melakukan suatu loncatan menuju "tujuan" bila OF=1.
l) JS (Jump On Sign) = Melakukan suatu loncatan menuju "tujuan" bila SF=1.

1.5 Addressing Mode

1.5.1 Pengertian dan Ciri Addressing Mode

Instruksi-instruksi Assembly ada yang membutuhkan operan baik satu maupun dua. Operan adalah data yang diolah atau dimanipulasi. Operan dibutuhkan oleh instruksi yang akan dijalankan. Misalnya, MOV AX,BX pada instruksi tersebut dibutuhkannya dua operan yaitu AX dan BX. Saat menjalankan suatu instruksi, prosesor bisa mendapatkan data atau operan dari register, dari dalam instruksi itu sendiri, atau dari suatu lokasi memori atau port I/O.

Terdapat tujuh macam mode pengalamatan, yaitu:

1. Register Addressing (Pengalamatan Register)
2. Immediate Addressing (Pengalamatan Segera)
3. Direct Addressing (Pengalamatan Langsung)
4. Register Indirect Addressing (Pengalamatan Tak-langsung Register)
5. Base Relative Addressing (Pengalamatan Relatif Dasar)
6. Direct Indexed Addressing (Pengalamatan Langsung Terindeks)

7. Base Indexed Addressing (Pengalamatan Dasar Terindeks)

Addresing Mode	Effective Address
Direct	6
Register Indirect	5
Register Relative	9
Based Indexed	
(BP)+(DI) or (BX)+(SI)	7
(BP)+(SI) or (BX)+(DI)	8
Based Indexed Relative	
BP)+(DI)+disp or (BX)+(SI)+disp	11
BP)+(SI)+disp or (BX)+(DI)+disp	12

Addressing Mode Table

Ciri-ciri addressing mode:

1) Direct = terdapat tanda kurung [] dan ada alamat offset, bukan alamat segment. Contoh: MOV AX,[37AD H].

2) Immediate = data tanpa kurung.

Contoh : MOV AX,37ADH

3) Relative = akan menggunakan register.

Contoh: MOV AL,[SI] alamat tidak diisikan langsung tetapi melalui register.

4) Based Indexed = yang terlibat adalah SI, dan DI.

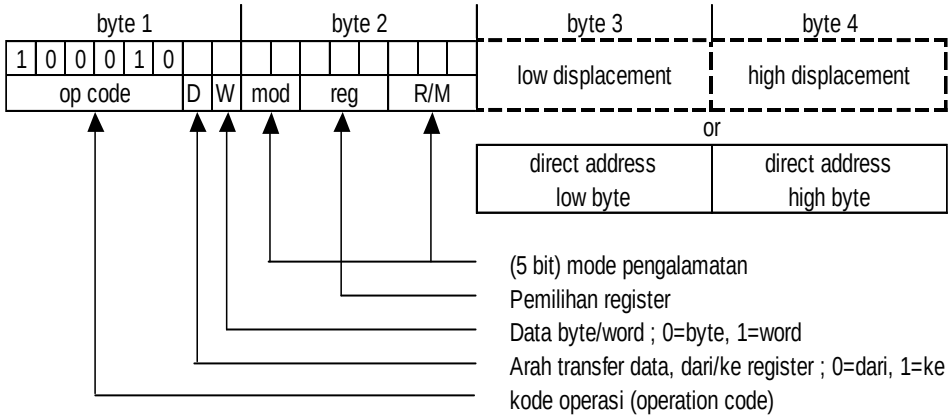
5) Based Indexed Relative = terdapat displacement.

Contoh program dari addressing mode:

Address	OP Code	Assembly/Mnemonic	Clock
CS:0100	B83412	MOV AX,1234H	4
CS:0103	353412	XOR AX,1234H	4
CS:0106	7402	JZ 010A	16 (branch)
CS:0108	B312	MOV BL,12	-(skipped)
CS:010A	8A0E3412	MOV CL,[1234]	8+6(EA)
CS:010E	88163412	MOV [1234],DL	9+6(EA)
		Total	53

1.5.2 OP Code

A. Kode Pola MOV



Konversi instruksi Bahasa assembly ke kode mesin

Byte 1

Op code : Kode Operasi (Operation code), 6 bits(seperti : add, subtract, mov)

D : Register direction bit. Menjelaskan arah transfer data dan/ ke register.
Bila register sebagai sumber [dari], maka D=0
Bila register sebagai tujuan [ke], maka D=1

W : Data byte/word. Menentukan apakah dilakukan operasi 8 atau 16 bit.
Bila data 8 bit, maka W=0
Bila data 16 bit, maka W=1

Byte 2

Mode filed : Mode pengalamatan (MOD)

Reg : Kode pemilihan register mengidentifikasi register pada operan 1

R/M : Kode pemilihan register/memori (R/M field) menentukan register/memori pada operan 2

LD/DLB : Low Displacement/Direct Address Low Byte

HD/DHB : High Displacement/ Direct Address High Byte

B. Kode Segment Register

Seg. Reg.	code
CS	01
DS	11
ES	00
SS	10

C. Pola MOD dan R/M

MOD / R/M	00	01	10	11	
				W=0	W=1
000	[BX] + [SI]	[BX] + [SI] + d8	[BX] + [SI] + d16	AL	AX
001	[BX] + [DI]	[BX] + [DI] + d8	[BX] + [DI] + d16	CL	CX
010	[BP] + [SI]	[BP] + [SI] + d8	[BP] + [SI] + d16	DL	DX
011	[BP] + [DI]	[BP] + [DI] + d8	[BP] + [DI] + d16	BL	BX
100	[SI]	[SI] + d8	[SI] + d16	AH	SP
101	[DI]	[DI] + d8	[DI] + d16	CH	BP
110	d16 direct address	[BP] + d8	[BP] + d16	DH	SI
111	[BX]	[BX] + d8	[BX] + d16	BH	DI
	MEMORY MODE			REGISTER MODE	

d8 : 8 - bit displacement, d16 : 16 - bit displacement

D. Kode Register 8 bit dan 16 bit

Register		code
W=0	W=1	
AL	AX	000
BL	BX	011
CL	CX	001
DL	DX	010
AH	SP	100
BH	DI	111
CH	BP	101
DH	SI	110

Pada OP CODE, terdapat memory mode, register mode, mod, dan R/M. Berikut ini adalah keterangannya:

1) R/M = register/memory
2) MOD = hanya ada 4, terdiri dari 2 bit (00, 01, 10, 11)
3) Register Mode = 11 (dipakai jika operand = register)
4) Memory Mode = 00 (dipakai jika tidak ada displacemet)
   01 (disp 8 bit)
   10 (disp 16 bit)

- [BP] = tidak bisa sendiri, yang ada adalah [BP+d8].
- W = 0 → kalau data 8 bit
- W = 1 → kalau data 16 bit
- D = direction

Direction terbagi menjadi bentuk from dan to. Berikut adalah penjelasannya:

MOV AL,CH	Transfer data dari CH ke AL
1 ← 0	CH = register from, AL = register to D=0 (FROM), D=1 (TO)

E. Mencari Op Code

1) MOV AX,BX

100010	0	1	1	1	0	1	1	0	0	0
	D	W	MOD		REG			R/M		

From:1000100111011000=89D8

To :1000101111000011=8BC3

2) MOV AL,[2400H]

100010	1	0	0	0	0	0	0	1	1	0
	D	W	MOD		REG			R/M		

Memory → Register : 1000101000000110 = 8A060024

3) ADD AX,BX

000000	0	1	1	1	0	1	1	0	0	0
	D	W	MOD		REG			R/M		

From : 0000000111011000 = 01D8

1.6 Instruksi Interrupt

1.6.1 Pengertian Interrupt

Interupsi adalah suatu permintaan khusus kepada mikroposesor untuk melakukan sesuatu. Bila terjadi interupsi, maka komputer akan menghentikan dahulu apa yang sedang dikerjakannya dan melakukan apa yang diminta oleh yang menginterupsi. Pada IBM PC dan kompatibelnya disediakan 256 buah interupsi yang diberi nomor 0 sampai 255. Nomor interupsi 0 sampai 1Fh disediakan oleh ROM BIOS, yaitu suatu IC didalam komputer yang mengatur

operasi dasar komputer. Jadi bila terjadi interupsi dengan nomor 0-1Fh, maka secara default komputer akan beralih menuju ROM BIOS dan melaksanakan program yang terdapat disana. Program yang melayani suatu interupsi dinamakan Interrupt Handler.

1.6.2 Perintah - perintah Interrupt

- INT 21h/AH=1

  Fungsi 1, register AH diisi dengan 1. Perintah ini akan membaca data dari input keyboard lalu akan ditampilkan di layar (karena dengan "echo"). Data input akan disimpan di register AL.

  Contoh perintah dengan fungsi 1 adalah sebagai berikut:

  MOV AH,1

  INT 21h

  Maka ketika keyboard menekan 'a' output pada layar adalah 'a' dan register AL akan diisi dengan nilai ascii dari 'a' yaitu 61.

- INT 21h/AH=2

  Fungsi 2, register AH diisi dengan 2. Perintah ini digunakan untuk menulis character yang akan ditampilkan di output. Input diisikan di register DL, lalu output ditampilkan di monitor di posisi yang tergantung kursor. Register AH = DL.

  Contoh perintah dengan fungsi 2 adalah sebagai berikut:

  MOV AH,2

  MOV DL,'a'

  INT 21H

  Maka pada output layar akan ditampilkan huruf a. Lalu pada register DL diisi dengan kode ascii dari 'a'.

- INT 21h/AH=7

  Fungsi 7, character input tanpa echo sehingga tidak akan ada output di monitor. Input diisikan ke register AL.

  Contoh perintah dengan fungsi 7 adalah sebagai berikut:

  MOV AH,7

  INT 21H

  Ketika keyboard ditekan 'a' maka AL akan diisi kode ascii nya yaitu 61. Akan tetapi tidak akan ada output di monitor.

- INT 21h/AH=9

  Fungsi 9, output berupa string yang disimpan di alamat CS:DX. String harus diakhiri dengan symbol $, alamat offset disimpan di register DX dan register AH akan diisi dengan 9. Contoh perintah dengan fungsi 9 adalah sebagai berikut:

  MOV DX,offset msg

  MOV AH,9

  INT 21H

  RET

  msg db “Hello World $”

  Ketika dilakukan run program, maka output yang akan dihasilkan adalah tulisan “Hello World” ditampilkan di monitor.
- INT 21h/AH=4Ch

  Fungsi stop program, mengembalikan kontrol ke operating system.

1.7 Overflow

Overflow merupakan kondisi ketika hasil operasi mengalami over capacity terhadap register atau memory areanya. Overflow dilihat dari range data. Berikut ini adalah range data 8 bit:

Data 8 bit tidak bertanda	Data 8 bit bertanda
0 - 255	(-128) – (+127)

Dalam penjumlahan decimal, overflow diketahui ketika hasil penjumlahan melebihi 8 bit, sehingga satu memory area tidak dapat menampungnya dan memerlukan area lebih. Berikut adalah contoh penjumlahan decimal dengan kondisi overflow.

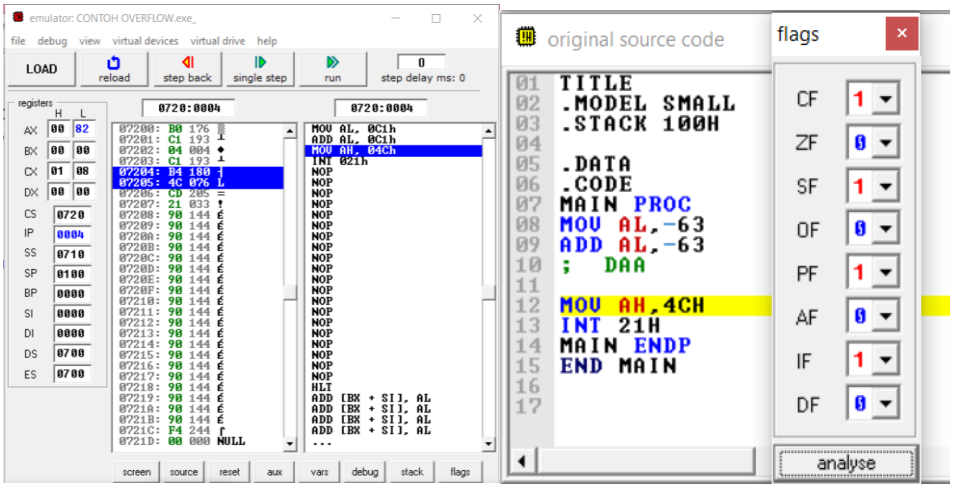
Data 1	Data 2	Hasil	Overflow
79	02	81	00
79	79	58	01

Untuk mengetahui suatu penjumlahan decimal mengalami overflow maka, bilangan diubah ke bentuk biner komplemen 2 terlebih dahulu lalu diubah ke heksadesimal

pada simulator. Berikut ini adalah contoh penjumlahan bilangan decimal yang tidak mengalami overflow pada simulator.

$-63_{(10)} = 1\ 0111111_{(2SM)} \rightarrow 1\ 1000000_{(K'1)} \rightarrow 1\ 1000001_{(K'2)} \rightarrow C1_{(16)}$

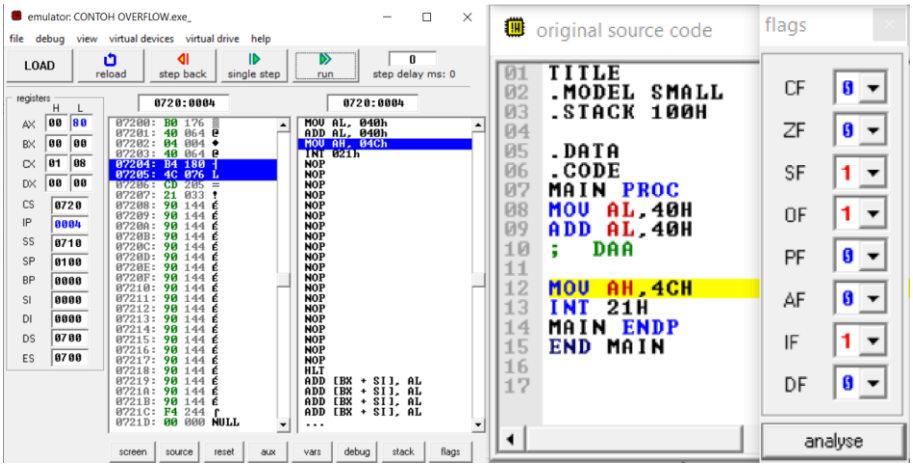
(-63) + (-63) = (-126)



Berikut ini adalah contoh penjumlahan bilangan hexadecimal yang mengalami overflow pada simulator.

$40H_{(16)} = 0\ 1000000_{(2)} \rightarrow 64_{(10)}$

64 + 64 = 128



Untuk mendeteksi overflow, dilakukan pada penjumlahan heksadesimal. Ketika OF dideteksi saat decimal maka nilai ZF = 1 (saat operasi = 0) tetapi instruksi transfer data tidak memengaruhi ZF karena zero flag hanya berpengaruh di ALU. Prosesor INTEL 8086 selalu menggunakan komplemen 2 atau bilangan bertanda. Untuk menggunakan perintah Jump, dibedakan menjadi dua, yaitu:

1. Signed Number
   JA & JB
2. Unsigned Number
   JG & JL

BAB II
PERINTAH PENJUMLAHAN

2.1 Penjumlahan Data Desimal

Pada prosesor intel 8086 jika ingin menjumlahkan suatu bilangan, hasilnya tidak bisa dalam bentuk desimal tetapi hasilnya dalam bentuk biner, karena prosesor intel 8086 merupakan prosesor biner.

Perintah penjumlahan tampil dalam berbagai format penulisan dalam beberapa jenis mikroprosesor. Instruksi penjumlahan dapat berupa ADD, ADC dan INC. ADD merupakan instruksi penjumlahan dimana carry tidak dimasukkan dalam perhitungan. Sedangkan ADC merupakan instruksi penjumlahan yang memasukkan carry dalam perhitungan. Inkremen (INC) adalah format khusus dari penjumlahan yaitu menambahkan suatu bilangan dengan satu. Instruksi penjumlahan terdiri dari 3 macam yaitu :

1. ADD, yaitu instruksi penjumlahan yang tidak menyertakan nilai carry dalam perhitungan. Format : SUB Source, Destination. Contoh : ADD AL, BL akan menghasilkan AL = AL + BL.
2. ADC, yaitu instruksi penjumlahan yang menyertakan nilai carry dalam perhitungan. Format : SBB Source, Destination. Contoh : ADC AL, BL akan menghasilkan AL = AL + BL + Carry.
3. INC, yaitu instruksi penjumlahan yang menambahkan dengan 1. Format : INC Destination. Contoh : INC AL akan menghasilkan AL = AL + 1.

Berdasarkan mode pengalamatan yang dipakai, maka instruksi penjumlahan dapat dibagi menjadi:

1. Penjumlahan register (register addition). Penjumlahan ini digunakan untuk menjumlahkan isi dari beberapa register. Contoh : ADD AL, BL akan menghasilkan AL = AL + BL. ADC AL, BL akan menghasilkan AL = AL + BL + Carry.
2. Penjumlahan segera (immediate addition). Penjumlahan ini digunakan untuk menjumlahkan isi register dengan nilai konstan. Contoh : ADD AL, 05H akan

menghasilkan AL = AL + 05H ADC AL, 05H akan menghasilkan AL = AL + 05H + Carry.

Penjumlahan decimal harus diberi faktor koreksi, yaitu penambahan 0110 (enam).

78
94 +
172

0111 1000
1001 0100 +
0001 0000 1100
0000 0110 0110

Jika lebih besar dari 9 maka menggunakan faktor koreksi

- DAA = memberi faktor koreksi agar hasil penjumlahan yang dilakukan benar secara decimal.
- Saat melakukan oeprasi menggunakan DAA hanya bias menggunakan AL.
- Saat melakukan operasi pada angka harus menggunakan H, karena jika tidak akan dianggap sebagai decimal dan akan mengkonversi ke biner.

  Contoh: [17H]
- Untuk data decimal wajib menggunakan DAA.

2.2 Penjumlahan Bilangan Oktal

Sistem Bilangan Oktal (Basis 8) adalah sistem bilangan yang mempunyai simbol angka (numerik) sebanyak 8 buah simbol, yaitu 0,1, 2, 3, 4, 5, 6, dan 7.Nilai suatu bilangan basis-8 dalam basis-10 dapat dinyatakan sebagai ∑(Nx8a). Dimana N = 0, 1, 2, 3, 4, 5, 6, atau 7.

- Bilangan octal hanya terdiri dari delapan angka (0,1,2,3,4,5,6,7)
- Penyebutan bilangan octal yaitu satuan, delapanan, enampuluh empatan, dst (kelipatan delapan). Contoh:

Enam puluh empatan ← 77 → satuan
46
145
↓
delapanan

- Apabila hasil penjumlahan 8 maka kembali ke 1. Contoh 7+1 = 10 adalah sebagai berikut:

  Contoh 1:

$$\begin{array}{r} 77 \\ 46 \\ \hline BD \\ 88 \\ \hline 145 \end{array}$$ → Karena D dan B lebih dari 7

Contoh 2:

$$\begin{array}{r} 12 \\ 12 \\ \hline 24 \\ 00 \\ \hline 24 \end{array}$$ → Karena 2 dan 4 tidak lebih dari 7

F0 atau 0F digunakan sebagai pembanding.

- Komplemen Ganjil (Komplemen 7)

  Komplemen 7 dapat dicari dengan cara tiap digit oktal dikurangkan pada angka tujuh.

  Contoh :

  Oktal = 425

  K'7 = 777 – 425 = 352

- Komplemen Genap (Komplemen 8)

  Komplemen 8 dapat dicari dari hasil penjumlahan K'7 dengan satu.

  Contoh :

  Oktal = 632

  K'7 = 777 – 632

  = 145

  K'8 = K'7 + 1

  = 145 + 1

  = 146

BAB III
KONVERSI DATA

3.1 Desimal ke Heksadesimal

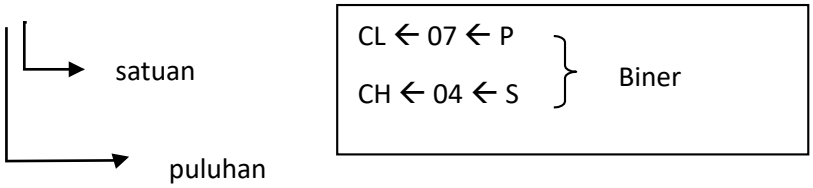
Cara konversi bilangan desimal ke octal adalah dengan membagi bilangan desimal dengan 16 dan menyimpan sisa bagi per seitap pembagian terus hingga hasil baginya < 16. Hasil konversi adalah urutan sisa bagi dari yang paling akhir hingga paling awal. Apabila sisa bagi diatas 9 maka angkanya diubah, untuk nilai 10 angkanya A, nilai 11 angkanya B, nilai 12 angkanya C, nilai 13 angkanya D, nilai 14 angkanya E, nilai 15 angkanya F.

Cara 1

Ex : $74_{(10)}$ = 74/16 = 4 sisa 10 (16) = 64/16 = 4 sisa 0

Cara 2

Ex : 74 = 7 * 10 +4



Keterangan : ketika satuan maka dikali satu, saat puluhan dikali 10 tetapi menggunakan perkalian binner

Contoh Penggandaan (A * 10) = A * (8+2)

5
5 +
10
10
20

saat 5 + 5 = 1 x penggandaan

saat 10 + 10 = 2 x penggandaan

dst

Konversi 4 digit

Ex : $9999_{(10)} = 270F_{(16)} 9876_{(10)} = 2694_{(16)}$

d1 = 9 d2 = 8 d3 = 7 d4 = 6

rumus : ((((d1 *A) +d2) *0A) +d3) *0A) +d4)

3.2 Heksadesimal ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan mengalikan satu persatu dengan bilangan 16 (basis hexa) pangkat 0 atau 1 atau 2 dst dimulai dari bilangan paling kanan. Kemudian hasilnya dijumlahkan. Misal, 79AF(hexa) = (Fx20) + (9x21) + (Ax22) = 15+144+2560+28672 = 31391(decimal). Berikut ini adalah cara mengkonversi data secara manual dari hexadecimal ke decimal.

A2(16) = 162 (10) → 2 x 160 = 2 → A x 161 = 160 ---------- + 160

Contoh:

A2 => 162

A2 = heksadesimal 16^1

162 = decimal 16^0

Penjumlahan = 2 x 16^0 = 2 x 1 = 2

A x 16^1 = 10 x 16 = 160

Jadi 2 + 160 =162

3.3 Oktal ke Heksadesimal

Teknik mengonversi bilangan octal ke hexa desimal adalah dengan mengubah bilangan octal menjadi biner kemudian mengubah binernya menjadi hexa. Ringkasnya octal→biner→hexa.

Untuk mengonversi data octal ke heksadesimal memiliki kondep yang hamper sama dengan mengonversi heksadesimal ke octal, perbedaannya terdapat pada banyaknya penggandaan, karena pada data octal memiliki basis 8 jadi diperlukan penggangdaan sebangyak 3 kaliagar di dapatkan faktor pengali 8 yaitu dengan menggunakan konsep : (A*8) = A* (2+2+2) yang dimana 2 merupakan penggandaan 1 kali. Kemudian ada cara lain yang bias dilakukan yaitu dengan cara menghapus bit 0 Contoh: 0111 0111 = 77 oktal, maka untuk mengubah ke heksadesimal yaitu dengan menghapus angka 0 sehingga menjadi 01111111 dan di dapatkan 3F untuk hasil pengonversian.

3.4 Heksadesimal ke Octal

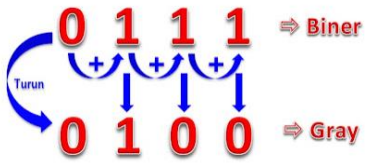
Begitu juga dengan konversi hexa desimal ke octal yakni dengan mengubah bilangan hexa ke biner kemudian diubah menjadi bilangan octal. Ringkasnya hexa->biner->octal.

Untuk mengonversi data dari heksadesimal ke octal kita harus metode dengan menyisipkan 0 pada setiap 3 bit bilangan untuk data heksadesimal yang berbentuk binner. Contoh : 3F(16) = 00111111 Kemudian susun bilangan per 3bit menjadi: 001 111 111 =077 Jadi untuk 3F(16) = 77(8) Kemudian agar bias dikonversikan ke bilangan octal maka perlu disisipkan 0 pada setiap 3bit sehingga menjadi : 000 0111 0111 = 77 ◊ nilai tetap dan tidak berubah. Untuk menyisipkan 0 digunakan perintah AND yang dimana 7 digunakan untuk mengambil 3 digit angka terakhir, yang kemudian digeser ke kanan dengan menggunakan perintah SHR. Dan yang terakhir kita harus meng AND kan dengan F8 agar 3 digit terakhir bernilai 0. Rumus yang digunakan 4 digit dalam 1 data heksadesimal yang akan dirubah ke octal adalah : ((((((d1*16)+d2)*16)+d3)*16)+d4).

3.5 Biner ke Gray

Gray code merupakan cerminan dari Binary code, artinya angka terakhir pada string dapat sama dengan angka awal tetap dalam urutan terbalik sehingga dapat memungkinkan untuk membangun dan meningkatkan kegunaan dari binner standart. Binary Code digunakan untuk mewakili teks, intruksi prosesor computer atau data lainnya menggunakan system dua symbol. Sistem symbol yang digunakan adalah 0 dan 1dari system angka biner. Kode biner memberikan pola digit biner atau dikenal sebagai bit. Dalam komputasi dan telekomunikasi kode biner digunakan untuk berbagi metode pengodean data, seperti string karakter mejadi string bit.

Contoh :



BAB IV
PERINTAH PERKALIAN DAN PEMBAGIAN

4.1 Perkalian Heksadesimal

Pada umumnya, perintah perkalian terbagi menjadi dua yaitu perkalian bilangan tak bertanda dan perkalian bilangan bertanda. Untuk bilangan bertanda, perintah perkalian yang digunakan adalah IMUL, sedangkan untuk bilangan tak bertanda digunakan perintah MUL. Pada data 8 bit dengan perintah MUL, proses perkalian terjadi di register AL lalu disimpan pada register AX. Sedangkan pada data 16 bit, proses perkalian terjadi di register AX lalu hasilnya disimpan pada register DX:AX. Hasil kali 8 bit akan selalu pada register AX, sedangkan hasil kali 16 bit akan selalu ada di register DX:AX.

4.1.1 Metode Penjumlahan Berulang

Pada penjumlahan berulang, diperlukan satu register counter dan satu register data. Metode ini kurang efektif untuk digunakan disebabkan oleh penggunaan pengali sebagai dasar dari proses perkalian. Ketika nilai pengali maksimum, maka dibutuhkan waktu yang lama dalam proses operasi perkaliannya.

4.1.2 Metode Shift and Add 1

Metode ini menggeser satu register yang dikali ke kiri dan menggeser satu register pengali ke kanan. Perkalian biner memiliki pengali 0 atau 1. Berikut ini contoh perkalian metode shift and add 1 pada bilangan decimal yang umum diketahui dan pada bilangan biner.

Desimal	Biner
49 73 ---------- x 147 343 . ---------- + 3577	1111 1010 ---------- x 0000 1111 . 0000 . 1111 . ---------- + 10010110

Misal, pada register CH diisi 0F dan CL diisi 0A. Maka berikut ini adalah cara perkaliannya menurut konsep shift and add 1.

CH digeser ke kiri ←									
CH	0	0	0	0	1	1	1	1	-
CL	0	0	0	0	1	0	1	0	CF
CL digeser ke kanan →									

CH	0	0	0	1	1	1	1	0	-
CL	0	0	0	0	0	1	0	1	0

Bagian yang ditandai kuning merupakan nilai biner yang masuk ke carry flag. Metode geser kanan, menyebabkan diperlukan pembanding pada CF karena CF akan memiliki 2 kemungkinan kondisi, yaitu 0 atau 1. Maka ketika CF bernilai 0, proses selanjutnya yang akan terjadi adalah nilai CH masuk ke register DL (penyimpan sementara). Sementara, ketika CF bernilai 1 maka CH akan digeser ke kiri.

4.1.3 Metode Shift and Add 2

Metode ini menggeser register pengali ke kiri dan menggeser register yang digunakan untuk menyimpan hasil kali ke kiri sementara nilai yang dikali tidak digeser. Misal, pada register CH diisi 0F dan CL diisi 0A. Maka berikut ini adalah cara perkaliannya menurut konsep shift and add 2.

CH	0	0	0	0	1	1	1	1	
CL	0	0	0	0	1	0	1	0	
---------- x									
DL	0	0	0	0	0	0	0	0	
	0	0	0	0	1	1	1	1	*
---------- +									
DL	**0**	**0**	**0**	**0**	**1**	**1**	**1**	**1**	++
	0	0	0	1	1	1	1	0	**
	0	0	1	1	1	1	0	0	**
	0	0	0	0	1	1	1	1	***
---------- +									

DL	**0**	**1**	**0**	**0**	**1**	**0**	**1**	**1**	++
	1	0	0	1	0	1	1	0	****

Ketika pengali bernilai 1, maka CH akan dijumlahkan ke DL, dan ketika pengali bernilai 0, maka DL digeser ke kiri. Warna merah menunjukkan proses yang terjadi saat pengali bernilai 1 pada proses pertama, warna oranye menunjukkan proses kedua saat pengali bernilai 0, begitu pun selanjutnya untuk warna kuning dan hijau. Hasil akhir diketahui bahwa 0F*0A = 96. Sesuai dengan hasil perkalian biner metode shift and add 2.

4.1.4 Metode MUL

Syntax perintah MUL adalah `MUL sumber` dengan sumber 8 atau 16 bit atau variabel. Jika sumber berukuran 8 bit seperti register BH, maka computer akan mengambil nilai pada BH dan nilai pada AL untuk dikalikan. Hasil akan disimpan di register AX. Jika sumber 16 bit seperti pada register BX, maka computer akan mengamnil nilai di register BX dan AX untuk dikalikan, hasil kali akan disimpan ke register DX:AX.

Misal, data pada memory area bernilai 6 digit ingin dikalikan. Datanya adalah abc*def. Maka untuk mempermudah, berikut ini adalah konsep perkalian dan penyimpanan hasil kalinya.

f*c					A	X
f*b						
f*a						
e*c						
e*b						
e*a						
d*c						
d*b						
d*a						
Final	[hasil+5]	[hasil+4]	[hasil+3]	[hasil+2]	[hasil+1]	[hasil+0]

Data pada memory area adalah [dat1+2] = a, [dat1+1] = b, [dat1+0] = c, [dat2+2] = d, [dat2+1] = e, [dat2+0] = f. Maka, setelah terkonsep seperti ini, prosesnya hanya perlu disesuaikan dengan tabel yang ada.

4.2 Pembagian Heksadesimal

4.2.1 Metode Pengurangan Berulang

Dengan satu register sebagai counter dan satu register sebagai data yang akan dikurangkan secara berulang. Namun, metode ini kurang efektif untuk digunakan karena waktu dari proses pembagian bergantung kepada pembagi. Jika pembagi bernilai minimum maka akan membutuhkan waktu yang lama untuk memproses perhitungan pembagian tersebut. Pembagian dengan metode ini terjadi dengan mekanisme seperti berikut:

19/4 = 19-4 = 15-4 = 11-4 = 7-4 = 3

Maka diketahui bahwa 19/4 adalah 4 dengan sisa bagi 3. Dilihat dari berapa kali bilangan dikurangi dengan 4 sehingga nilainya lebih kecil dari 4. Jumlah bilangan dikurangi 4 adalah sebanyak 4 kali dengan sisa hasil bagi nya adalah 3.

4.2.2 Metode Shift and Subtract

Metode ini menggeser bit data yang dibagi ke kanan sampai bisa dikurangi oleh pembagi.

4.2.3 Metode DIV

Perintah ini memiliki syntax DIV sumber dengan sumber data register 8 atau 16 bit atau variabel. Perintah ini untuk membagi nilai pada register AX secara otomatis, oleh karena itu data yang akan dibagi harus dipindahkan ke AX setelah itu dilakukan perintah DIV.

4.2.4 Metode Komplemen 10

Pembagian dengan metode ini terjadi dengan mekanisme seperti berikut:

- Tinjau nilai pembagi dari komplemen 10.
- Nilai yang dibagi ditambahkan dengan nilai pembagi dalam komplemen 10.

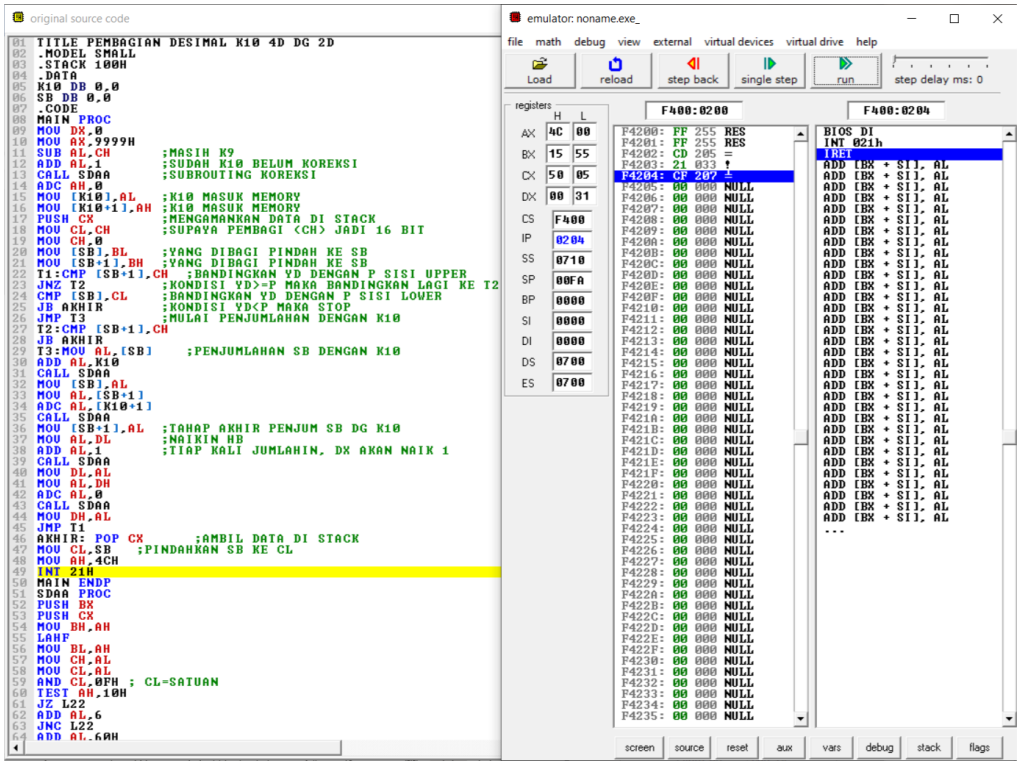
K'9 dari 04 adalah → 99-04 = 95.

K'10 dari 95 adalah → 95+1 = 96. Maka 96 adalah K'10 dari 04.

Pembagian terjadi seperti berikut:

19/4 = 19+96 = 115+96 = 111+96 = 107+96 = 103

Nilai 96 dijumlahkan sebanyak 4 kali, maka hasil bagi adalah 4 sedangkan sisa baginya adalah 03. Angka 1 yang diberi highlight kuning merupakan nilai carry. Berikut ini adalah contoh penyelesaian program untuk membagi data desimal 4 digit yang ada pada reg BX dengan data desimal 2 digit y gada pada reg CH. Hasil bagi disimpan pada reg DX dan sisa bagi pada reg CL. Metode yang digunakan: komplemen 10.




```
SDAA PROC
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL,0FH ; CL=SATUAN
     TEST AH,10H
     JZ L22
     ADD AL,6
     JNC L22
     ADD AL,60H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,60H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,9
     JBE L44
     ADD AL,6
     JNC L44
     ADD AL,60H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,90H
     JBE L55
     ADD AL,60H
     OR BL,1
L55: MOV AH,BL
     SAHF
     MOV AH,BH
     POP CX
     POP BX
     RET
   SDAA ENDP
END MAIN
```

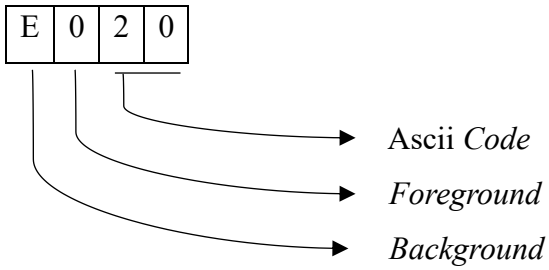
BAB V
PERINTAH MENGGAMBAR

EMU8086 adalah emulator perangkat lunak untuk prosesor Intel 8086 yang digunakan untuk mengembangkan program di lingkungan Windows. EMU8086 memiliki kemampuan untuk mengeksekusi kode program yang ditulis dalam bahasa assembly Intel 8086. Dalam menggambar pada simulator EMU8086, penting untuk diingat bahwa lokasi memori di layar dipetakan dalam mode grafis. Hal ini mengartikan bahwa untuk menulis ke layar, programmer harus menulis ke alamat memori yang sesuai dengan posisi piksel pada layar. Selain itu, mode tampilan ke mode grafis pun perlu diubah sebelum menggambar pada layar.

Jika menyimpan sesuatu pada alamat *segment* 0B800, maka akan berimbas ke *monitor*. Terdapat dua *attribute* yaitu warna *background* dan warna *foreground*, kedua perintah ini dilihat dari *colour coding*. Berikut ini adalah code-code warna atau *colour codes*.

0 = Hitam	6 = Coklat	C = Merah Muda
1 = Biru	7 = Abu Muda	D = Magenta Muda
2 = Hijau	8 = Abu Tua	E = Kuning
3 = Cyan	9 = Biru Muda	F = Putih
4 = Merah Tua	A = Hijau Muda	
5 = Magenta	B = Cyan Muda	

Berikut ini adalah contoh aplikasi kode warna untuk warna *background* kuning.



41
E0

Perintah E020 disimpan di data area 0B800:0000.
Maka, dari perintah tersebut akan dimunculkan *output monitor* huruf A dengan warna *font* hitam dan *background* kuning.

Data Area = DS:SI dengan DS adalah alamat *segment* dan SI adalah alamat *offset*.

Extra Area = ES:DI dengan ES adalah alamat *segment* dan DI adalah alamat *offset*.

Perintah MOVSB → data area akan menjadi *source* atau sumber, sedangkan extra area menjadi tujuan.

Terdapat dua arah untuk memindahkan data area ke extra area yang dapat dilihat dari *direction flag*, yaitu:

1) Arah 0 = Memindahkan data dengan alamat segment yang sesuai, tetapi alamat offsetnya berurut dari yang terkecil ke terbesar. DF = 0
2) Arah 1 = Memindahkan data dengan alamat segment yang sesuai, tetapi alamat offsetnya berurut dari yang terbesar ke terkecil. DF = 1

Perintah-perintah yang terkait dengan pemindahan data area ke extra area adalah:

1) MOVSB → Copy DS:[SI] ke ES:[DI]
2) CLD → DF = 0
3) STD → DF = 1
4) REP → repeat, berkaitan dengan CX. Jika CX = 5 maka perintah MOVSB akan berulang sebanyak 5 kali. CX akan menjadi *counter* pengulangan.

BAB VI
PROGRAM KALKULATOR SEDERHANA

Dengan menggunakan EMU8086 kita bisa memanfaatkannya menjadi kalkulator sederhana satu digit dan dua digit. Pada program kalkulator sederhana, data-data yang diinputkan melalui keyboard dihitung menurut operasi penjumlahan, pengurangan, perkalian, atau pembagian. Data input akan dioperasikan sesuai dengan input symbol dari user. Ketika user menginputkan symbol tambah, maka program akan mengoperasikan perintah-perintah pada procedure tambah. Begitupun ketika user menginputkan symbol kurang, kali, dan bagi. Pada program kalkulator ini, procedur bagi disimpan pada main procedure sehingga ketika pengecekan symbol pada bagian input data ditemukan symbol bagi, program akan menjalankan perintah pada bagian main procedure.

Contoh program kalkulator sederhana satu digit:

```
TITLE  KALKULATOR DES 1 DGT
.MODEL SMALL
.STACK 100H
.DATA
MEM DB 2 DUP(0)
DAT DB 0
TAMB DB 0
KUR DB 0
KAL DB 2 DUP(0)
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
A0:  MOV AH,2
     MOV DL,0DH ;PINDAHKAN CURSOR
     INT 21H    ;KE BARIS BARU
     MOV DL,0AH
     INT 21H
P0:  MOV DL,20H ;CTK 1 SPASI
     MOV AH,2
     INT 21H
     CALL INKB   ;AMBIL DATA1

     MOV CL,DH
     CALL INKB   ;AMBIL DATA2
     MOV CH,DH

     MOV DL,0
     MOV DH,CL
N1:  CMP DH,CH
     JB N2

     MOV AL,DH
     SUB AL,CH
     DAS
     MOV DH,AL
     MOV AL,DL
     ADD AL,1
     DAA
     MOV DL,AL
     JMP N1

N2:  MOV BL,DL
     MOV BH,DL  ;BUAT DISIMPAN DATA ASLINYA
     CMP BL,9
     JBE N3
     AND BL,0F0H
     MOV CL,4
     SHR BL,CL
     ADD BL,30H
     MOV DL,BL
     MOV AH,2
     INT 21H
N3:  AND BH,0FH
     ADD BH,30H
     MOV DL,BH
     MOV AH,2
     INT 21H
     MOV DL,','
     MOV AH,2
     INT 21H

     MOV SI,3 ; UNTUK 3 DGT DIBELAKANG KOMA
     MOV BL,DH
     MOV BH,0
     MOV CL,4
     SHL BX,CL
     MOV DH,0
     MOV DL,CH
     MOV [MEM],BL
     MOV [MEM+1],BH
K0:  MOV CH,0
K1:  MOV AL,[MEM]
     MOV AH,[MEM+1]
     CMP AX,DX
     JB K2
     MOV AL,[MEM]
     SUB AL,DL
     DAS
     MOV [MEM],AL
     MOV AL,[MEM+1]
     SBB AL,DH
     MOV [MEM+1],AL

     MOV AL,CH
     ADD AL,1
     DAA
     MOV CH,AL
     JMP K1
K2:  PUSH DX
     MOV DL,CH
     ADD DL,30H
     MOV AH,2
     INT 21H
     POP DX
     MOV AL,[MEM]
     MOV AH,[MEM+1]
     CMP AX,0
     JE A0
     MOV CL,4
     SHL AX,CL
     MOV [MEM],AL
     MOV [MEM+1],AH
     DEC SI
     JNZ K0
     JMP A0                          ;loop

   MOV AH,4CH
   INT 21H
   MAIN ENDP
INKB PROC
 S1:   PUSH CX
       MOV CH,[DAT]
       MOV AH,7    ;DH=DATA1 ATAU DATA2
       INT 21H
```

```
        CMP AL,30H
        JB S1
        CMP AL,39H
        JA S1
        MOV AH,2     ;CTK DGT1
        MOV DL,AL
        INT 21H      ;-----------
        SUB AL,30H
        MOV DH,AL    ;DH=DGT1

        MOV AH,7
        INT 21H
UJI: TEST CH,1
        JE GNP
GJL:  CMP AL,'='
        JE T1
        MOV AH,7
        INT 21H
        JMP GJL
T1:   MOV AH,2
        MOV DL,AL    ;CTK '='
        INT 21H
        MOV BL,AL
        INC [DAT]
        JMP P55
GNP:  CMP AL,'/'
        JNE T2
        CALL CETAK
        JMP P55
   T2:CMP AL,'x'
        JNE T3
        CALL CETAK
        CALL KALI
        JMP P55
   T3:CMP AL,'+'
        JNE T4
        CALL CETAK
        CALL TAMBAH
        JMP P55
   T4:CMP AL,'-'
        JNE T6
        CALL CETAK
        CALL KURANG
        JMP P55
   T6:CMP AL,'^'
        JNE T7
        CALL CETAK
  ;     CALL PANGKAT
        JMP P55
   T7:CMP AL,'!'
        JNE T5
        CALL CETAK
        ;CALL FAKTORIAL
        JMP P55
   T5:MOV AH,7
        INT 21H
        JMP GNP
        CALL CETAK
```

```
   P55:POP CX
        RET
  INKB ENDP

CETAK PROC
      MOV AH,2
      MOV DL,AL    ;CTK '/'
      INT 21H
      INC [DAT]
      RET
      CETAK ENDP

TAMBAH PROC
    MOV [TAMB],DH
    CALL INKB
    MOV AL,[TAMB]
    ADD AL,DH
    DAA
    MOV DH,AL
    MOV DL,20H
    MOV AH,2
    INT 21H
    CMP DH,9 ;periksa jml digit
    JBE LUP
    MOV AH,2
    MOV DL,31h;Tampilkan Puluhan
    INT 21H
LUP:AND DH,0FH
    ADD DH,30H
    MOV DL,DH
    MOV AH,2
    INT 21H

    JMP A0                                  ;loop
    TAMBAH ENDP

KURANG PROC
    MOV [KUR],DH
    CALL INKB
    MOV AL,[KUR]
    SUB AL,DH
    CMP AL,0F7H
    JAE LAP
    MOV DH,AL
    JMP LIP

LAP:MOV AL,DH
    SUB AL,[KUR]
    DAS
    MOV DH,AL
    MOV AH,2
    MOV DL,2Dh;Tampilkan NEGATIF
    INT 21H

LIP:MOV DL,20H
    MOV AH,2
    INT 21H
```

```
    AND DH,0FH
    ADD DH,30H
    MOV DL,DH ;Tampilkan Satuan
    MOV AH,2
    INT 21H

    JMP A0                                  ;loop
    KURANG ENDP

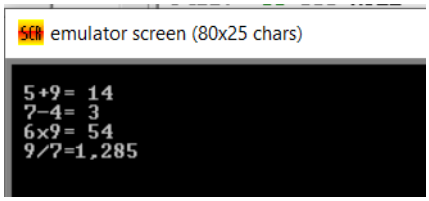
KALI PROC
    MOV [KAL+0],DH
    CALL INKB
    MOV [KAL+1],DH
    MOV CH,[KAL+0]
    MOV CL,[KAL+1]
    MOV AL,CH    ; AL = 0C8h
    MOV BL,CL
    MUL BL
    MOV BH,AL
    MOV CH,AL
    AND CH,0X0F
    MOV AL,CH
    ADD AL,0H
    DAA
    MOV CH,AL;CH=SATUAN
    MOV DL,BH
    AND DL,0XF0
    SHR DL,1
    SHR DL,1
    SHR DL,1
    SHR DL,1
    MOV AL,DL
    ADD AL,0H
    DAA
    MOV DL,AL
    MOV BL,4
L1:MOV AL,DL
    ADD AL,AL
    DAA
    MOV DL,AL
    ADC DH,DH   ;PENGGANDAAN HARUS DENGAN DH
    DEC BL
    JNZ L1
    MOV AL,DL
    ADD AL,CH
    DAA
    MOV DH,AL

    MOV DL,20H
    MOV AH,2
    INT 21H
    CMP DH,9 ;periksa jml digit
    JBE L2
    MOV AH,2
    MOV BL,DH
    AND BL,0XF0
    SHR BL,1
    SHR BL,1
```

```
    SHR BL,1
    SHR BL,1
    ADD BL,30H
    MOV DL,BL;Tampilkan Puluhan
    INT 21H
L2: MOV AH,DH
    AND AH,0FH
    ADD AH,30H
    MOV DL,AH ;Tampilkan Satuan
    MOV AH,2
    INT 21H

    JMP A0                                  ;loop
    KALI ENDP
END  MAIN
```

Hasil yang ditampilkan oleh monitor:



Contoh program kalkulator sederhana dua digit:

```
TITLE  KALKULATOR DES 2 DGT
.MODEL SMALL
.STACK 100H
.DATA

MEM     DB 2 DUP(0)
STR     DB 'KALKULATOR 2 DIGIT$'
DAT     DB 0
DAT1    DW 0
KAL     DB 2 DUP(0)
TAMB    DB 2 DUP(0)
KUR     DB 2 DUP(0)

.CODE
MAIN PROC
     MOV AX,@DATA
         MOV DS,AX
         MOV DX, OFFSET STR
         MOV AH, 9
         INT 21H
   AW1: MOV AX,0
         MOV DX,0
         MOV CX,0
         MOV BX,0
   A0:  MOV AH,2
         MOV DL,0DH ;PINDAHKAN CURSOR
         INT 21H    ;KE BARIS BARU
         MOV DL,0AH
         INT 21H
   P0:  MOV DL,20H ;CTK 1 SPASI
         MOV AH,2
         INT 21H
         CALL INKB   ;AMBIL DATA1

         MOV CL,DH
         CALL INKB   ;AMBIL DATA2
         MOV CH,DH

         MOV DL,0
         MOV DH,CL
   N1:   CMP DH,CH
         JB N2

         MOV AL,DH
         SUB AL,CH
         DAS
         MOV DH,AL
         MOV AL,DL
         ADD AL,1
         DAA
         MOV DL,AL
         JMP N1

   N2:   MOV BL,DL
         MOV BH,DL   ;BUAT DISIMPAN DATA ASLINYA
         CMP BL,9
         JBE N3
         AND BL,0F0H
         MOV CL,4
         SHR BL,CL
         ADD BL,30H
         MOV DL,BL
         MOV AH,2
         INT 21H
   N3:   AND BH,0FH
         ADD BH,30H
         MOV DL,BH
         MOV AH,2
         INT 21H
         MOV DL,','
         MOV AH,2
         INT 21H

         MOV SI,3 ; UNTUK 3 DGT DIBELAKANG KOMA
         MOV BL,DH
         MOV BH,0
         MOV CL,4
         SHL BX,CL
         MOV DH,0
         MOV DL,CH
         MOV [MEM],BL
         MOV [MEM+1],BH
   K0:   MOV CH,0
   K1:   MOV AL,[MEM]
         MOV AH,[MEM+1]
         CMP AX,DX
         JB K2
         MOV AL,[MEM]
         SUB AL,DL
         DAS
         MOV [MEM],AL
         MOV AL,[MEM+1]
         SBB AL,DH
         MOV [MEM+1],AL

         MOV AL,CH
         ADD AL,1
         DAA
         MOV CH,AL
         JMP K1
   K2:   PUSH DX
         MOV DL,CH
         ADD DL,30H
         MOV AH,2
         INT 21H
         POP DX
         MOV AL,[MEM]
         MOV AH,[MEM+1]
         CMP AX,0
         JE A0
         MOV CL,4
         SHL AX,CL
         MOV [MEM],AL
         MOV [MEM+1],AH
         DEC SI
         JNZ K0
         JMP A0
    MOV AH,4CH
    INT 21H
    MAIN ENDP
```

```
INKB PROC
 S1:     PUSH CX
         MOV CH,[DAT]
         MOV AH,7
         INT 21H
         CMP AL,30H
         JB S1
         CMP AL,39H
         JA S1
         MOV AH,2        ;CTK DGT1
         MOV DL,AL
         INT 21H         ;-----------
         SUB AL,30H
         MOV DH,AL       ;DH=DGT1
         MOV AH,7        ;BACA KB KE 2
         INT 21H
         CMP AL,30H
         JB UJI
         CMP AL,39H
         JA UJI
         MOV AH,2
         MOV DL,AL
         INT 21H         ;CTK DGT2
         MOV CL,4
         SHL DH,CL
         MOV CL,AL
         SUB CL,30H
         ADD DH,CL
         MOV AH,7
         INT 21H
  UJI: TEST CH,1
         JE GNP
  GJL:   CMP AL,'='
         JE T1
         MOV AH,7
         INT 21H
         JMP GJL
   T1:   MOV AH,2
         MOV DL,AL       ;CTK '='
         INT 21H
         MOV BL,AL
         INC [DAT]
         JMP P55
  GNP:   CMP AL,'/'
         JNE T2
         CALL CETAK
         JMP P55
      T2:CMP AL,'x'
         JNE T3
         CALL CETAK
         CALL KALI
         JMP AW1
      T3:CMP AL,'+'
         JNE T4
         CALL CETAK
         CALL TAMBAH
         JMP AW1
```

```
      T4:CMP AL,'-'
         JNE T5
         CALL CETAK
         CALL KURANG
         JMP AW1
      T5:MOV AH,7
         INT 21H
         JMP GNP
         CALL CETAK
    P55:POP CX
         RET
   INKB ENDP

CETAK PROC
       MOV AH,2
       MOV DL,AL    ;CTK 'OPERATOR'
       INT 21H
       INC [DAT]
       RET
       CETAK ENDP

KALI PROC
     MOV AX,0
     MOV BX,0
     MOV CX,0

 L0:MOV [KAL+0],DH
     CALL INKB
     MOV [KAL+1],DH
     MOV CH,[KAL]
     MOV CL,[KAL+1]

     MOV AX,0
     MOV DX,0

     MOV BX,CX
     AND CL,0FH
     CMP CL,0
     JE L3
     CMP CL,1
     JE L2
     MOV AL,CH
 L1:ADD AL,CH
     DAA
     ADC AH,0
     MOV DL,AL
     MOV AL,AH
     ADD AH,0 ;CURIGA
     DAA
     MOV DH,AH
     MOV AX,DX
     DEC CL
     CMP CL,1
     JNZ L1
     JMP L3

 L2:MOV DL,CH
```

```
 L3:MOV [DAT1],DX

    MOV CX,BX
    MOV AX,0
    MOV DX,0

    MOV BX,CX
    MOV AL,CH
    AND CL,0F0H
    SHR CL,4

    CMP CL,0
    JE L6
    CMP CL,1
    JE L5

 L4:ADD AL,CH
    DAA
    ADC AH,0
    MOV DL,AL
    MOV AL,AH
    ADD AH,0     ;CURIGA
    DAA
    MOV DH,AH
    MOV AX,DX
    DEC CL
    CMP CL,1
    JNZ L4
    JMP L6

 L5:MOV DL,CH

 L6:SHL DX,4
    MOV AX,DX
    ADD AX,[DAT1]
    DAA
    ADC AH,0
    MOV DL,AL
    MOV AL,AH
    ADD AL,0
    DAA
    MOV DH,AL

    MOV CX,DX

    MOV DL,CH
    AND DL,0F0H
    SHR DL,4
    ADD DL,30H
    MOV AH,2
    INT 21H

    MOV DL,CH
    AND DL,0FH
    ADD DL,30H
    MOV AH,2
    INT 21H
```

```
    MOV DL,CL
    AND DL,0F0H
    SHR DL,4
    ADD DL,30H
    MOV AH,2
    INT 21H

    MOV DL,CL
    AND DL,0FH
    ADD DL,30H
    MOV AH,2
    INT 21H

    RET
    KALI ENDP

TAMBAH PROC

    MOV [TAMB+0],DH
    CALL INKB
    MOV [TAMB+1],DH
    MOV CH,[TAMB+0]
    MOV CL,[TAMB+1]
    MOV BX,0

    MOV AL,CH
    ADD AL,CL
    DAA
    ADC BH,0
    MOV BL,AL
    MOV DL,20H
    MOV AH,2
    INT 21H
    MOV DL,20H
    MOV AH,2
    INT 21H

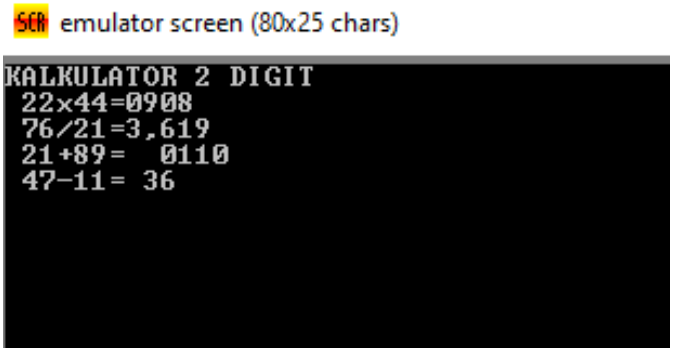
    MOV DL,BH
    AND DL,0F0H
    SHR DL,4
    ADD DL,30H
    MOV AH,2
    INT 21H

    MOV DL,BH
    AND DL,0FH
    ADD DL,30H
    MOV AH,2
    INT 21H

    MOV DL,BL
    AND DL,0F0H
    SHR DL,4
    ADD DL,30H
    MOV AH,2
    INT 21H
```

```
351
352     MOV DL,BL
353     AND DL,0FH
354     ADD DL,30H
355     MOV AH,2
356     INT 21H
357     RET
358     TAMBAH ENDP
359
360 KURANG PROC
361     MOV [KUR+0], DH
362     CALL INKB
363     MOV [KUR+1],DH
364     MOV CH,[KUR+0]
365     MOV CL,[KUR+1]
366
367     MOV AL,CH
368     SUB AL,CL
369     DAS
370     MOV DH,AL
371     JMP LIP
372
373 LIP:MOV DL,20H
374     MOV AH,2
375     INT 21H
376
377
378     MOV DL,DH
379     AND DL,0F0H
380     SHR DL,4
381     ADD DL,30H
382     MOV AH,2
383     INT 21H
384
385     MOV DL,DH
386     AND DL,0FH
387     ADD DL,30H
388     MOV AH,2
389     INT 21H
390     RET
391     KURANG ENDP
392
393 END  MAIN
```

Hasil yang ditampilkan oleh monitor:



BAB VII
LOOK-UP TABLE METHOD

Metode ini merupakan metode satu-satunya yang bisa digunakan untuk menyelesaikan permasalahan ketika pada nilai data input tidak ada hubungannya dengan data output. Seperti contohnya program cash-register pada minimarket atau warung sederhana yang menggunakan computer. Metode Look-Up Table memiliki konsep seperti array, namun pada bahasa assembly data disimpan di memory data. Input yang diberikan akan dianggap sebagai indeks yang selalu diketahui sebagai bilangan heksadesimal. Sedangkan data output dari tiap indeks yang diinputkan disimpan dalam stack. Dibawah ini merupakan contoh dari look-up table. Contoh:

Input	Output
0	38
1	07
2	93
3	15
4	5

Look-up table merupakan array yang digunakan untuk menyimpan nilai atau konstanta yang akan digunakan dalam program. Dalam bahasa assembly ini, look-up table didefinisikan di bagian data area menggunakan directive DB, DW, atau DD tergantung pada jenis data yang disimpan dalam array. Untuk mengakses tiap elemennya, digunakan label yang telah dideklarasikan di bagian code segment dan menambahkan nilai indeks pada awal alamat array yang disimpan dalam register. Cara penggunaan look-up tabel dapat bervariasi tergantung pada kebutuhan program yang dibuat seperti pada program cash register, bilangan berpangkat, atau bilangan factorial.

Caranya dengan menggunakan array, kemudian simpan di memory sebagai table data

Index A [0] = 38;

B [1] = 7;

C [2] = 93;

Contoh tabel:

Kode barang	Harga barang	Nama barang
0	4.000	Buku tulis A
1	12.000	Tipe-ex
2	6.500	Penghapus
3	350.000	Kalkulator
4	525.000	Tas gendong
5	400.000	Sepatu
6	4.000	Kaos kaki
7	3.500	Pulpen
8	8.000	Penggaris A
9	15.000	Penggaris B
A	10.000	Spidol
B	425.000	Jam Tangan A
C	75.000	Jam tangan B
D	500	Permen

Langkah pertama tentukan digit terbanyak yang ada, karena output paling besar yang menentukan berapa jumlah lokasi memori, untuk semua bilangan yang ada termauk yang terkecil, contoh apabila bilangan yang paling banyak digitnya memeerlukan 3 lokasi memori sedangkan terkecil memerlukan 1 lokasimemori maka kita harus menggunakan 3 untuk semua jenis digit. Kemudian untuk cara cepat pengurangan bilangan ascii yaitu kurang semua dengan 30H apabila lebih dari 9 maka kurangi lagi dengan 7

0D → memulai diawal

0A → untuk memberi jarak (enter)

Berikut ini adalah contoh program look-up table cash register.

```
TITLE METODA LOOK-UP TABLE "CASH REGISTER"
.MODEL SMALL
.STACK 100H
.DATA
TABEL DB 0,40H,0,0,20H,1
      DB 0,65H,0,0,0,35H
      DB 0,50H,52H,0,0,40H
      DB 0,40H,0,0,35H,0
      DB 0,80H,0,0,50H,1
      DB 0,0,1,0,50H,42H
      DB 0,50H,7,0,5,0
DGT   DB 4,5,4,6,6,6,4,4,4,5,5,6,5,3
      ; DGT= JML DGT OUTPUT
.CODE
MAIN  PROC
      MOV AX,@DATA
      MOV DS,AX
R1:   MOV AH,7
      INT 21H
      CMP AL,2FH
      JBE R1
      CMP AL,39H
      JBE R2
      CMP AL,41H
      JB R1
      CMP AL,45H
      JAE R1
R2:   MOV BH,AL
      MOV DL,AL
      MOV AH,2
      INT 21H
      MOV DL,3DH   ; cetak =
      MOV AH,2
      INT 21H
      MOV DL,'R'
      MOV AH,2
      INT 21H
      MOV DL,'p'
      MOV AH,2
      INT 21H
      MOV DL,'.'
      MOV AH,2
      INT 21H
      MOV AL,BH
      SUB AL,30H
      CMP AL,9
      JBE R3
      SUB AL,7
R3:   MOV BL,AL   ;NILAI INPUT DI BL
      MOV DH,3
      MUL DH
      MOV AH,0
      MOV SI,AX
      MOV BH,0
      MOV DI,BX
      MOV DH,DGT+DI ; DH=JML CETAK
      CMP DH,6
      JZ C6
      CMP DH,5
      JZ C5
      CMP DH,4
      JZ C4
      CMP DH,3
      JZ C3
C6:   MOV CL,TABEL+SI+2
      CALL SUB2
C5:   MOV CL,TABEL+SI+2
      CALL SUB1
C4:   MOV CL,TABEL+SI+1
      CALL SUB2
C3:   MOV CL,TABEL+SI+1
      CALL SUB1
      MOV CL,TABEL+SI
      CALL SUB2
      CALL SUB1
      MOV DL,0DH
      MOV AH,2
      INT 21H
      MOV DL,0AH
      MOV AH,2
      INT 21H
      JMP R1
      MOV AH,4CH
      INT 21H
MAIN  ENDP

SUB2    PROC
        PUSH CX ;MENCETAK PULUHAN DARI CH
        MOV CH,CL
        AND CH,0F0H
        SHR CH,1
        SHR CH,1
        SHR CH,1
        SHR CH,1
        ADD CH,30H
        MOV DL,CH
        MOV AH,2
        INT 21H
        POP CX
        RET
SUB2    ENDP
SUB1    PROC
        PUSH CX ;MENCETAK SATUAN DARI CH
        MOV CH,CL
        AND CH,0FH
        ADD CH,30H
        MOV DL,CH
        MOV AH,2
        INT 21H
        POP CX
        RET
SUB1    ENDP
END     MAIN
```

BAB VIII
INTERFACING I/O DAN MEMORY

8.1 Interfacing I/O

8.1.1 I/O Instructions

- Tipe instruksi : transfer dan membaca informasi dari I/O device. Instruksi transfer data antara I/O device dan mikroprosesor accumulaltor (AL, AX, EAX) dikenal dengan IN dan OUT.
- I/O address disimpan di register DX (16 bit).
- 8-bit form (p8) = fixed address → disimpan di ROM disertai dengan instruksinya.
- 16-bit form = variable address → disimpan di register DX.
- Data ditransfer melalui instruksi IN atau OUT lalu I/O address (port number) muncul di address bus. p8 muncul di address bus dengan koneksi A7-A0 dengan bit A15-A8 (sama dengan 00000000_2). Sementara untuk >A15 maka tidak dapat didefinisikan.
- Range lokasi decoder dari 16-bit address bus adalah 0000H-03FFH.
- Alamat instruksi INS dan OUTS yang menggunakan register DX tidak mentransfer data dari accumulator ke I/O device, tapi mentransfer data antara memory dengan I/O device.
- Alamat memory dari instruksi INS dan OUTS adalah ES:DI untuk INS, sedangkan untuk OUTS adalah DS:DI.

8.1.2 Isolated dan Memory-Mapping I/O

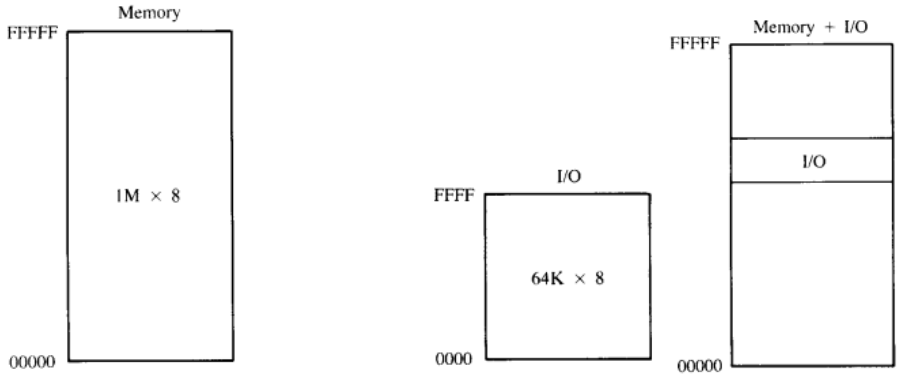
Isolated I/O

- Port = alamat-alamat untuk isolated I/O. Dipisahkan oleh memory sehingga pengguna dapat memperluas memory hingga maksimal tanpa menggunakan memory space.
- Kekurangan : data ditransfer antara I/O, akses mikroprosesor harus melalui instruksi IN, INS, OUT, OUTS.

- Separate Control Signal = pengembangan dari M/IO dan W/R dimana mengindikasikan bahwa sebuah I/O membaca IORC atau menulis IOWC operasi.
- Sinyal mengindikasikan bahwa port address sedang digunakan untuk menyeleksi I/O device.
- Pada PC, isolated I/O port digunakan untuk mengontrol peripheral devices.
- 8-bit port → untuk mengakses device pada system board (seperti timer dan keyboard interface)
- 16-bit port → untuk mengakses serial/paralel ports (video dan disk drive system).

Memory-Mapping I/O

- Menggunakan banyak instruksi yang mentransfer data antara mikroprosesor dan memory.
- Memory-Mapping device diperlakukan seperti lokasi memory di memory map.
- Keuntungan : setiap instruksi transfer memory bisa digunakan untuk mengakses I/O device, IORC/IOWC signals tidak memilliki fungsi apapun di memory-mapping I/O system, dan bisa mengurangi nilai kebutuhan arus untuk decoder.
- Kerugian : porsi memory sistem digunakan sebagai I/O map.

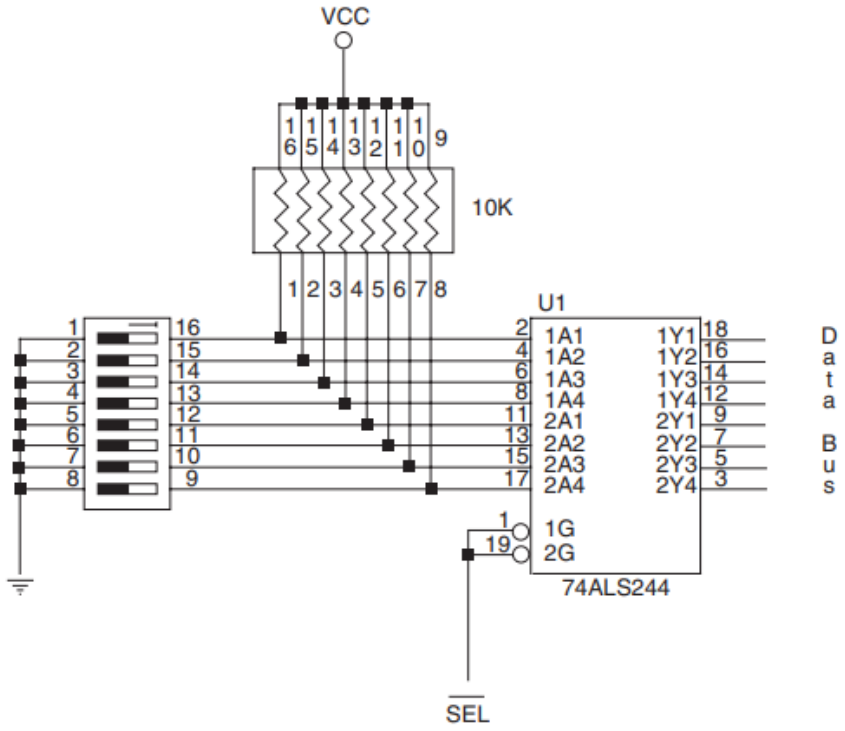


(a) (b)

- Gambar (a) menunjukkan isolated I/O, sedangkan gambar (b) menunjukkan Memory-Mapping I/O.

8.1.3 Basic Input Interface

- Terdiri dari 3-state buffers yang dibangun dari 8-bit input port.
- Gambar dibawah ini menunjukkan bahwa mikroprosesor memiliki kemampuan untuk membaca 8 input yang didapat dari switch yang menghubungkan ke setiap 8-bit sections dari data bus ketika sinyal SEL bernilai logic 0.

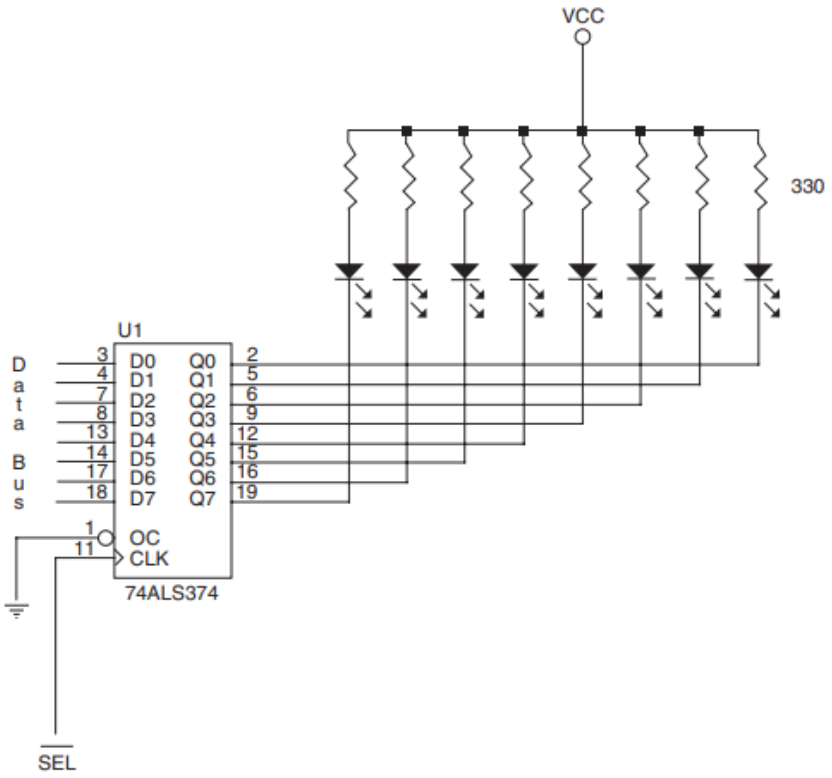


- Ketika mikroprosesor mengeksekusi instruksi IN, I/O port address akan melakukan decode untuk memproses logic 0 di SEL. Nilai logic 0 di bagian output control inputs 1G/2G dari 74ALS244 buffer akan mengakibatkan koneksi data input (A) terhubung dengan data output (Y).
- Jika logic 1 ditempatkan di outout control inputs, maka device akan memasuki mode 3-state dengan impedansi tinggi yang otomatis akan memutuskan koneksi dari switch dan data bus.

8.1.4 Basic Output Interface

- Terdiri dari latches yang menerima data dari mikroprosesor dan biasanya harus menyimpannya untuk beberapa device external.
- Latches (flip-flop) menyimpan nilai output melalui mikroprosesor dari data bus sehingga LED bisa menyala dengan berapapun nilai biner 8-bit.

- Ketika instruksi OUT dieksekusi, maka data dari AL, AX, atau EAX akan ditransfer ke latch melalui data bus. Input di D terhubung ke data bus lalu akan melakukan capture output data. Output Q di latch akan terikat ke LED.
- Q = 0 → LED menyala.
- OUT → SEL aktif → data output to latch captured.



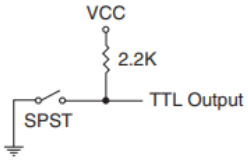
8.1.5 Handshaking

- Metode ini menyinkronkan I/O device dengan mikroprosesor.
- Contoh perangkat yang memerlukan handshaking : printer paralel yang mencetak beberapa ratus karakter per detik (CPS).
- Handshaking di interface I/O terjadi ketika perangkat I/O mengirimkan sinyal ke komputer atau sistem lainnya untuk mengindikasikan bahwa perangkat I/O siap menerima data atau mengirim data ke komputer atau sistem lainnya. Komputer atau sistem lainnya kemudian mengirimkan sinyal balik ke perangkat I/O untuk mengindikasikan bahwa data telah diterima atau bahwa perangkat I/O dapat mengirimkan data ke komputer atau sistem lainnya.
- Proses handshaking ini biasanya terjadi dalam beberapa langkah, termasuk pengiriman sinyal permintaan dari perangkat I/O ke komputer atau sistem

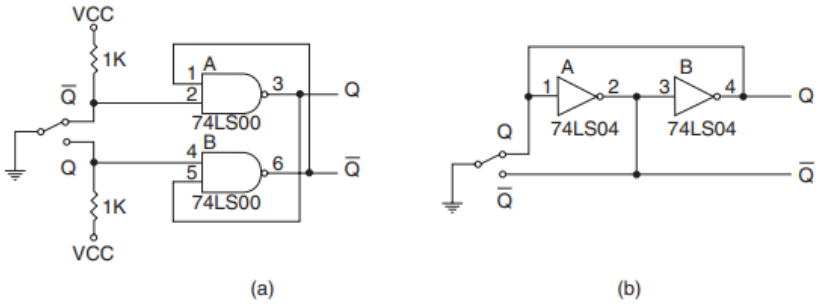
lainnya, pengiriman sinyal persetujuan dari komputer atau sistem lainnya ke perangkat I/O, pengiriman data dari perangkat I/O ke komputer atau sistem lainnya, dan pengiriman sinyal konfirmasi dari komputer atau sistem lainnya ke perangkat I/O bahwa data telah diterima.

8.1.6 Input Device

- Perangkat input yang sudah TTL dan kompatibel dapat dihubungkan ke mikroprosesor dan komponen penghubungnya atau berbasis sakelar.
- Perangkat berbasis sakelar terbuka atau terhubung tidak memiliki level TTL yang jelas seperti logika 0 atau logika 1.
- Untuk mengubah perangkat berbasis sakelar menjadi perangkat input yang kompatibel dengan TTL, digunakan resistor pull-up untuk memastikan sinyal keluaran berada pada level logika 1.



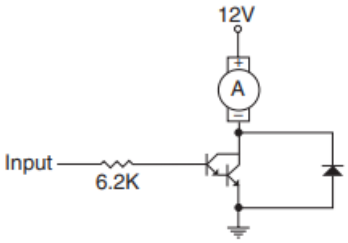
- Kontak sakelar mekanis cenderung memantul saat ditutup, yang dapat menyebabkan masalah jika digunakan sebagai sinyal clocking untuk rangkaian digital. Untuk mengatasi masalah tersebut, dua sirkuit penghilang bouncing dapat dibangun. Sirkuit pertama memerlukan lebih banyak uang untuk dibuat, sedangkan sirkuit kedua lebih praktis dan lebih murah karena hanya memerlukan dua inverter, bukan dua gerbang NAND dan tidak memerlukan resistor pull-up.
- Kedua sirkuit yang digambarkan pada gambar dibawah ini adalah flip-flop asinkron, dan sirkuit kedua berfungsi dengan mengingat negara flip-flop sampai kontak sakelar bergerak pertama kali menyentuh sambungan Q. Hal ini menghilangkan pantulan dan memastikan bahwa sinyal input yang diterima akurat.



- Gambar (a) merupakan conventional debouncing, gambar (b) merupakan practical debouncing.

8.1.7 Output Device

- Sebelum menghubungkan perangkat keluaran, kita harus memahami tegangan dan arus dari mikroprosesor atau komponen antarmuka TTL yang keluar.
- Output perangkat jauh lebih beragam daripada input.
- Driver transistor dan inverter TTL adalah antarmuka LED sederhana yang digunakan untuk menghubungkan LED ke pin periferal mikroprosesor.
- Darlington-pair digunakan untuk menghubungkan motor DC ke mikroprosesor dan dapat menangani arus hingga 4A.
- Resistor pembatas arus harus digunakan dalam kedua antarmuka dan nilainya harus dipilih sesuai dengan arus keluaran yang diinginkan.
- Dioda harus digunakan untuk mencegah kerusakan pada Darlington-pair oleh bantingan induktif dari motor.
- Nilai resistor pembatas arus dan nilai resistor bias harus dipilih dari nilai standar yang tersedia.



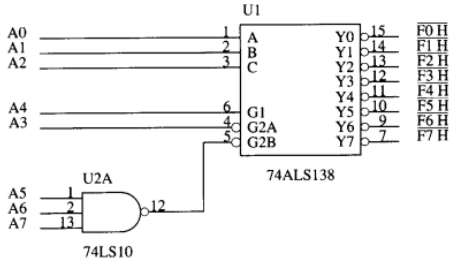
- Gambar diatas merupakan DC motor yang di-interface-kan ke sistem menggunakan Darlington-pair.

8.1.8 I/O Port Addressing Decoding

- Decoding alamat port I/O dan decoding alamat memori sangat mirip, terutama untuk perangkat I/O yang dipetakan memori. Perbedaan utama antara keduanya adalah nomor pin alamat yang terhubung ke decoder.
- Untuk memori, didekode A31–A0, A23–A0, atau A19–A0, sedangkan untuk isolated I/O didekode A15–A0. Dalam sistem komputer pribadi, decode dilakukan pada semua port address 16 bit I/O.
- Perbedaan lain dengan isolated I/O adalah IORC dan IOWC mengaktifkan I/O devices untuk membaca dan menulis.

Decoding 8-bit I/O Port Addresses

- Instruksi I/O tetap menggunakan alamat port I/O 8 bit, yaitu 0000H-00FFH pada A15-A0.
- Jika sistem tidak akan berisi lebih dari 256 perangkat I/O, koneksi alamat A7-A0 seringkali cukup untuk mendekode alamat port I/O 8 bit.
- Register DX juga dapat mengakses port I/O 00H-FFH.
- PC tidak pernah menggunakan atau mendekode alamat 8 bit.
- Decoder 74ALS138 dapat digunakan untuk mendekode port I/O 8 bit F0H hingga F7H, dan PLD (Programmable Logic Device) adalah rangkaian decoder yang lebih efisien karena hanya menggunakan satu perangkat.

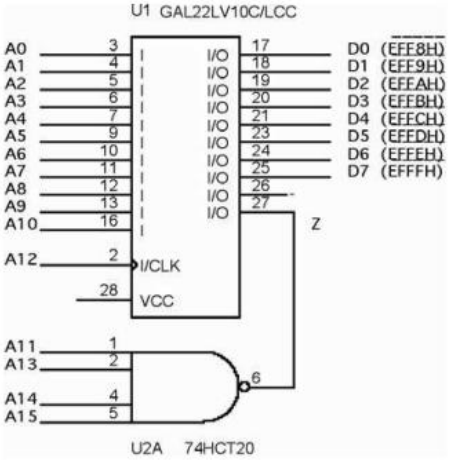


- Port decoder yang mendekode port I/O 8-bit. Decoder ini mengenerate output secara active low untuk port F0H-F7H.

Decoding 16-bit I/O Port Addresses

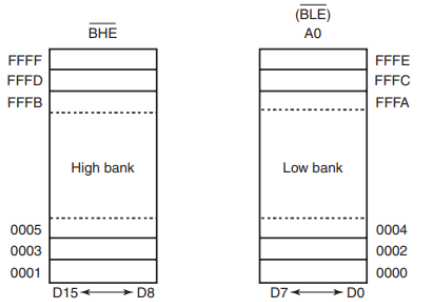
- Sistem PC menggunakan alamat I/O 16-bit, sedangkan sistem tertanam biasanya menggunakan alamat I/O 8-bit.
- Decoding alamat I/O 16-bit memerlukan delapan baris alamat tambahan (A15-A8) untuk didekodekan.

- Gerbang NAND 4-input digunakan untuk menerjemahkan bagian dari alamat (A15, A14, A13, dan A11) karena PLD tidak memiliki input alamat yang cukup.
- Output dari gerbang NAND terhubung ke input Z dari PLD dan diterjemahkan sebagai bagian dari alamat port I/O.



8.1.9 8 and 16-bit Wide I/O Ports

- Alamat port I/O lebih mudah untuk dipecahkan kode daripada alamat memori karena jumlah bitnya yang lebih sedikit.
- Data ditransfer ke perangkat I/O yang lebar 8 atau 16 bit dan terdapat 64K port 8-bit yang berbeda pada sistem 16-bit seperti 80386SX.
- Sistem I/O pada mikroprosesor 16-bit seperti 80386SX terdiri dari dua bank memori 8-bit yang terpisah.
- Setiap penulisan I/O 8-bit memerlukan strobo tulis terpisah untuk berfungsi dengan benar dan pembacaan I/O tidak memerlukan strobo baca terpisah.
- Port I/O menggunakan alamat 8-bit dan dapat dialamatkan sebagai port 8-bit terpisah atau bersama-sama sebagai satu port 16-bit.



- Gambar 11-14 mengilustrasikan sistem yang berisi dua perangkat keluaran 8-bit yang berbeda, yang terletak di alamat I/O 8-bit 40H dan 41H.
- Karena ini adalah perangkat 8-bit dan karena mereka muncul di bank I/O yang berbeda, sinyal tulis I/O terpisah dihasilkan untuk mencatat sepasang kait yang menangkap data pelabuhan.
- Semua port I/O menggunakan alamat 8-bit. Dengan demikian, port 40H dan 41H masing-masing dapat dialamatkan sebagai port 8-bit terpisah, atau bersama-sama sebagai satu port 16-bit.

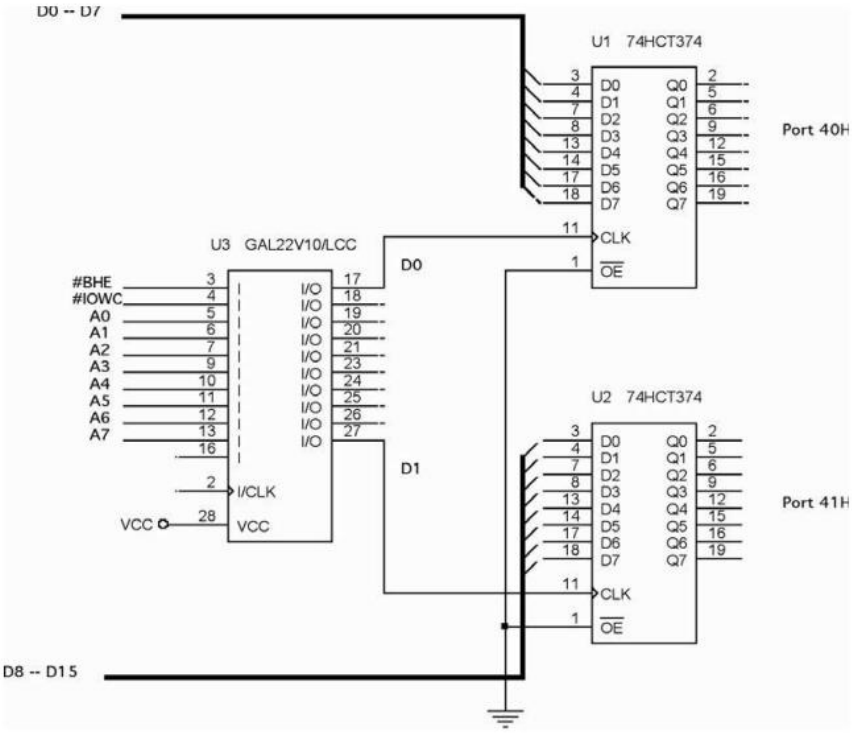
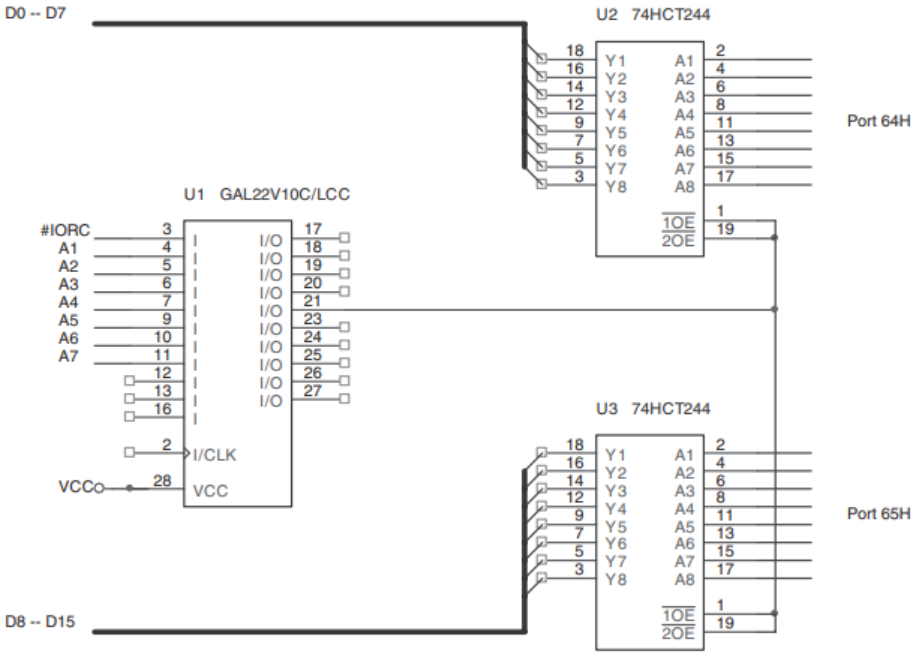


FIGURE 11–14 An I/O port decoder that selects ports 40H and 41H for output data.

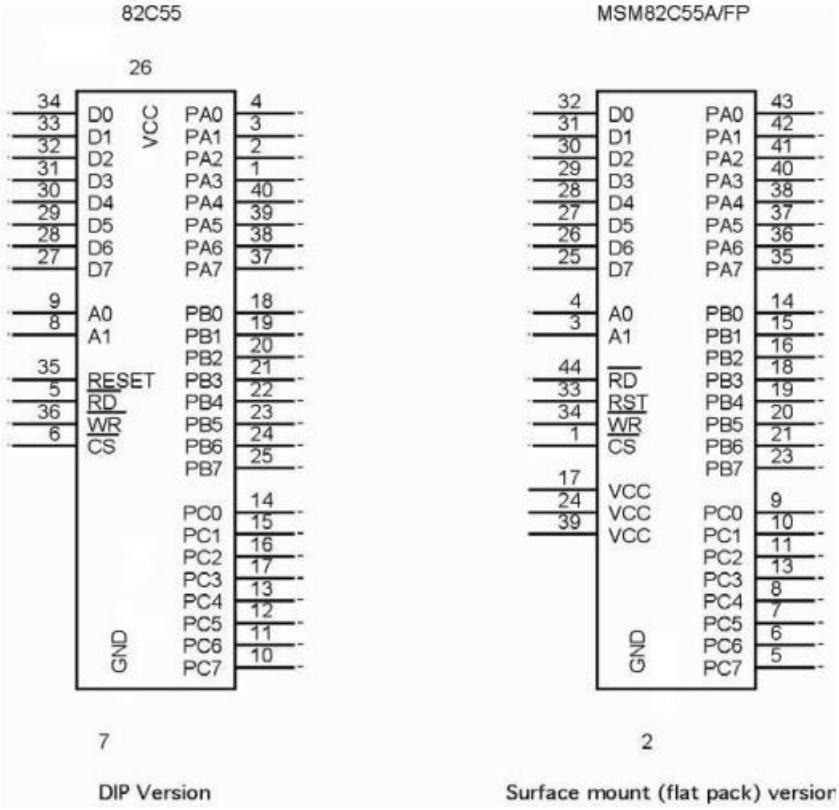
- Saat memilih 16-bit wide I/O devices, BLE dan BHE pins tidak memiliki fungsi karena kedua I/O banks dipilih bersamaan. Gambar dibawah ini mengilustrasikan 16-bit input device yang terhubung ke fungsi 8-bit I/O address 64H dan 65H. PLD decoder tidak memiliki koneksi ke address bit BLE dan BHE karena sinyalnya tidak diperuntukkan untuk 16-bit-wide I/O devices.



8.1.10 Programmable Peripheral Interface

- 82C55 adalah interfacing component yang banyak digunakan dalam banyak aplikasi dengan 24 pin I/O yang dapat diprogram dalam kelompok 12 pin.
- PPI dapat beroperasi dalam tiga mode operasi yang berbeda dan dapat berinteraksi dengan perangkat I/O yang kompatibel dengan TTL.
- 82C55 masih banyak digunakan meskipun sudah ada perangkat lain yang lebih canggih untuk aplikasi sederhana.
- 82C55 (versi CMOS) memerlukan insert wait states jika dioperasikan dengan mikroprosesor yang menggunakan clock lebih tinggi dari 8 MHz.
- 82C55 digunakan sebagai interface untuk keyboard dan port printer paralel pada banyak komputer pribadi dan terdapat sebagai fungsi dalam chip antarmuka yang juga mengontrol timer dan membaca data dari antarmuka keyboard.

8.1.11 Basic Description of the 82C55



- Diagram pin-out 82C55 digambarkan dalam format DIP dan permukaan mount dengan tiga port I/O yang diprogram sebagai grup A, B, dan C.
- 82C55 dipilih oleh pinnya untuk pemrograman dan akses ke port, dengan daftar pilihan dicapai melalui A1 dan A0 pin input.
- 82C55 dapat dibaca atau ditulis dengan input berlogika 0 dan alamat I/O yang benar diterapkan pada A1 dan A0 pin.
- 82C55 terhubung ke 80386SX sehingga berfungsi pada I/O 8-bit alamat port C0H (port A), C2H (port B), C4H (port C), dan C6H (register perintah).
- Masukan RESET ke 82C55 menginisialisasi perangkat setiap kali mikroprosesor diatur ulang.
- Setelah RESET, tidak ada perintah lain diperlukan untuk memprogram 82C55 jika digunakan sebagai perangkat input untuk ketiga port tersebut.
- 82C55 digunakan untuk berbagai perangkat pada komputer pribadi, seperti keyboard kontrol, speaker, timer, dan port printer paralel pada port I/O 378H–37BH.

8.1.12 Programming The 82C55

- 82C55 diprogram melalui dua register perintah internal.
- Pin grup B diprogram sebagai input atau output dan beroperasi dalam mode 0 atau mode 1.
- Pin grup A dapat beroperasi pada mode 0, 1, dan 2.
- Byte perintah B mengatur atau mereset bit port C hanya jika 82C55 diprogram dalam mode 1 atau 2.
- Fungsi bit set/reset digunakan untuk mengatur atau menghapus bit kontrol pada port C dan tidak mempengaruhi pin port C lainnya.

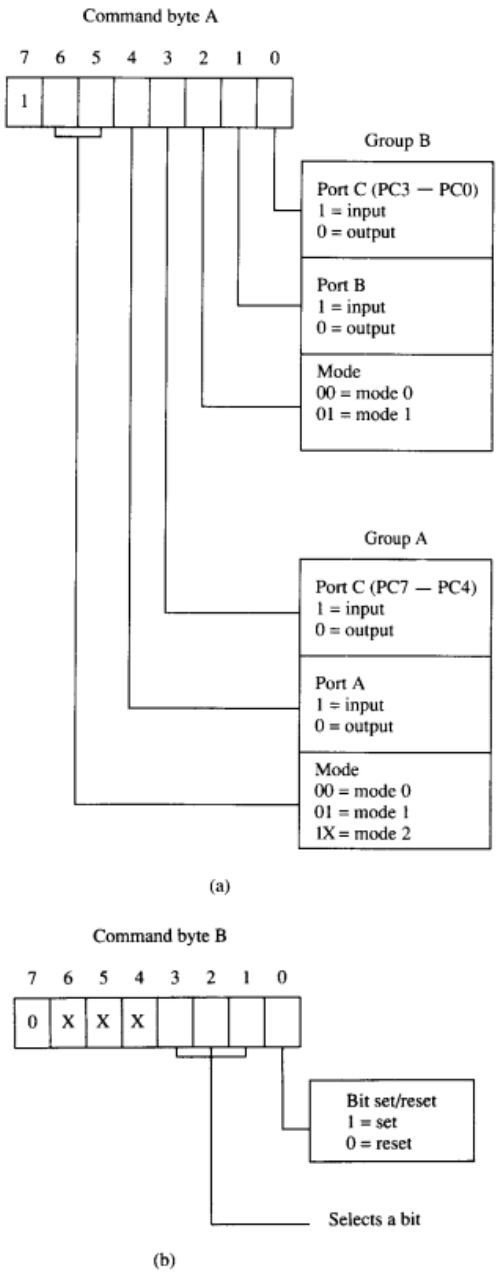
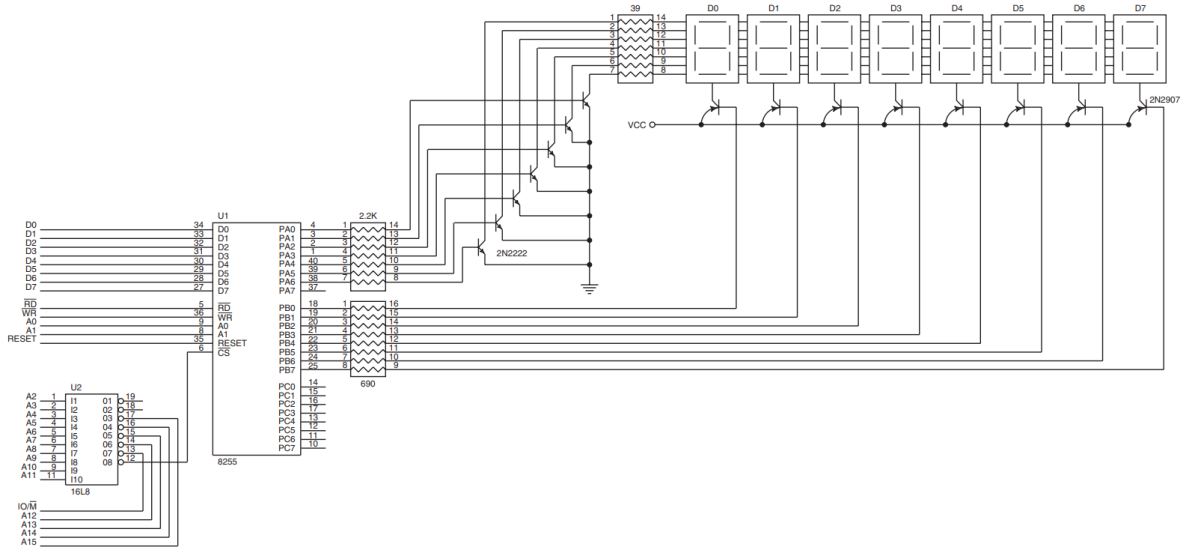


FIGURE 11–20 The command byte of the command register in the 82C55. (a) Programs ports A, B, and C. (b) Sets or resets the bit indicated in the select a bit field.

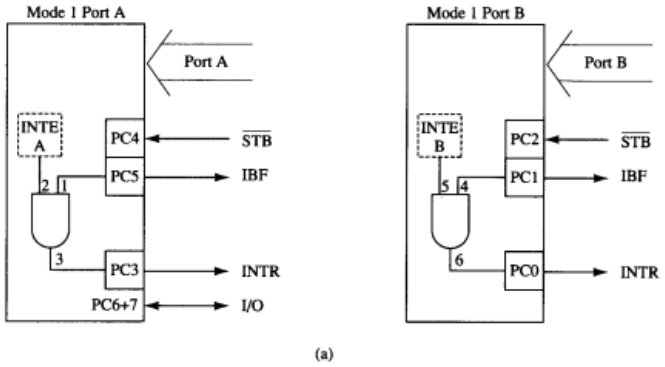
Mode 0 Operation

- Operasi mode 0 pada 82C55 berfungsi sebagai perangkat input buffer atau pengunci perangkat keluaran.
- 82C55 dapat terhubung dengan satu set delapan layar LED tujuh segmen untuk menampilkan data.
- Port A dan B diprogram sebagai (mode 0) port output terkunci sederhana.
- Port A menyediakan input data segmen ke display, sedangkan port B menyediakan sarana untuk memilih satu posisi display pada satu waktu untuk multiplexing.
- 82C55 dihubungkan ke mikroprosesor 8088 melalui PLD dan berfungsi pada nomor port I/O 0700H-0703H.



Mode 1 Strobed Input

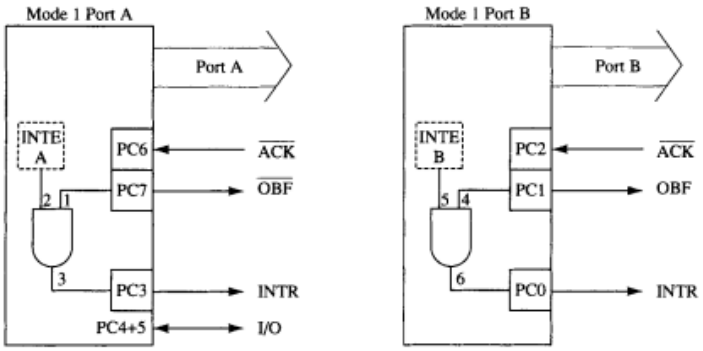
- Operasi mode 1 menyebabkan port A dan/atau port B berfungsi sebagai perangkat input pengunci. Ini memungkinkan data eksternal disimpan ke dalam port sampai mikroprosesor siap untuk mengambilnya.
- Port C digunakan dalam operasi mode 1—bukan untuk data, tetapi untuk sinyal kontrol atau handshaking yang membantu mengoperasikan salah satu atau kedua port A dan port B sebagai port input yang berstrobe.
- Operasi output berstrobe mirip dengan operasi output mode 0, kecuali bahwa sinyal kontrol disertakan untuk menyediakan handshaking.



(a)

Mode 1 Strobed Output

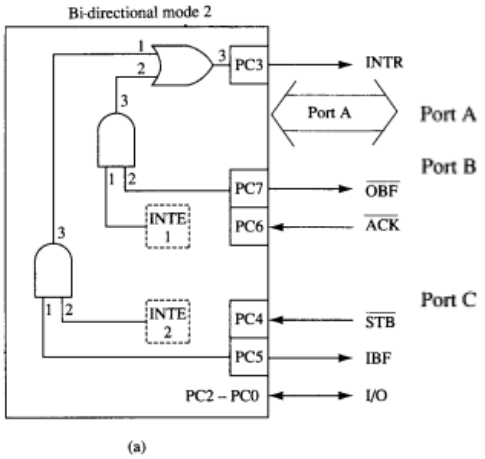
- Operasinya mirip dengan mode 0, sinyal control disertakan untuk handshaking.



(a)

Mode 2 Bidirectional Operation

- Hanya diperbolehkan adalah grup A, port A menjadi dua arah, memungkinkan data dikirim dan diterima melalui delapan kabel yang sama.
- Data bus dua arah berguna saat menghubungkan dua komputer.



(a)

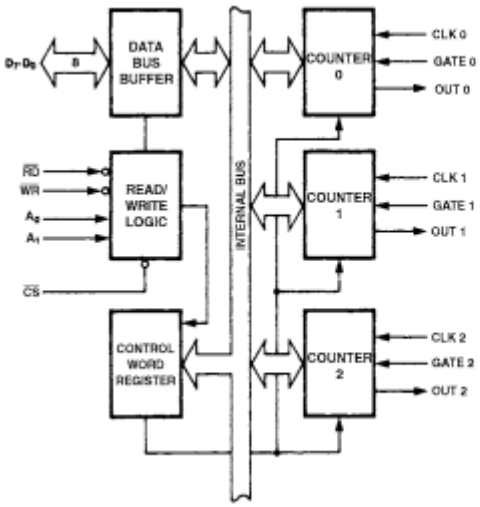
	Mode 0		Mode 1		Mode 2
	IN	OUT	IN	OUT	I/O
	IN	OUT	IN	OUT	Not used
0	IN	OUT	$INTR_B$	$INTR_B$	I/O
1			IBF_B	$\overline{OBF}_B$	I/O
2			$\overline{STB}_B$	$\overline{ACK}_B$	I/O
3			$INTR_A$	$INTR_A$	INTR
4			$\overline{STB}_A$	I/O	$\overline{STB}$
5			IBF_A	I/O	IBF
6			I/O	$\overline{ACK}_A$	$\overline{ACK}$
7			I/O	$\overline{OBF}_A$	$\overline{OBF}$

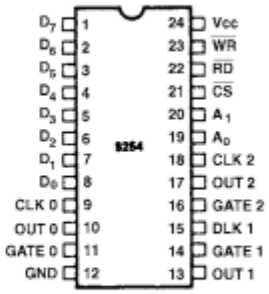
8254 Programmable Interval Timer

- Terdiri dari tiga pencacah (timer) 16-bit yang dapat diprogram.
- Frekuensi input maksimum yang diizinkan untuk penghitung adalah 10 MHz.
- Timer ini juga muncul di PC yang didekodekan pada port 40H–43H untuk menghasilkan interupsi pengatur waktu dasar yang terjadi pada sekitar 18,2 Hz, menyegarkan sistem memori DRAM, dan memberikan sumber pengaturan waktu ke speaker internal dan perangkat lain. Timer di PC adalah 8253 bukan 8254.

8254 Functional Description

- Sinyal yang terhubung ke mikroprosesor adalah pin bus data (D7-D0), RD, WR, CS, dan input alamat A1 dan A0.
- Input alamat hadir untuk memilih salah satu dari empat register internal yang digunakan untuk pemrograman, membaca, atau menulis ke counter.

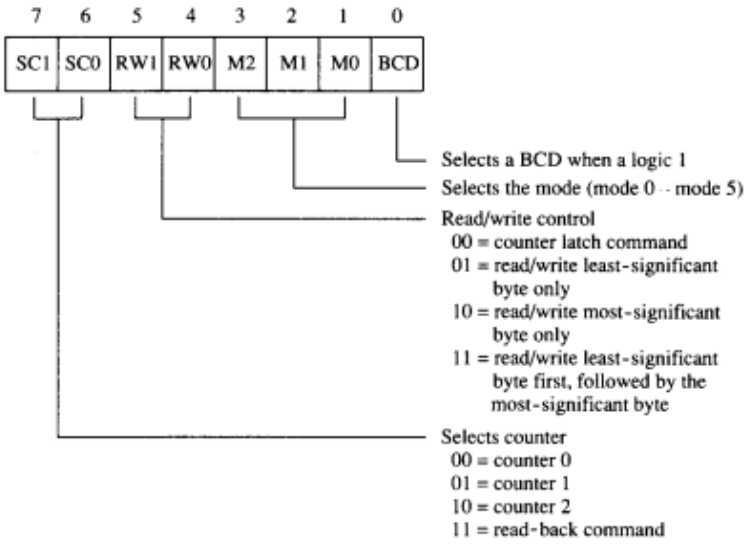




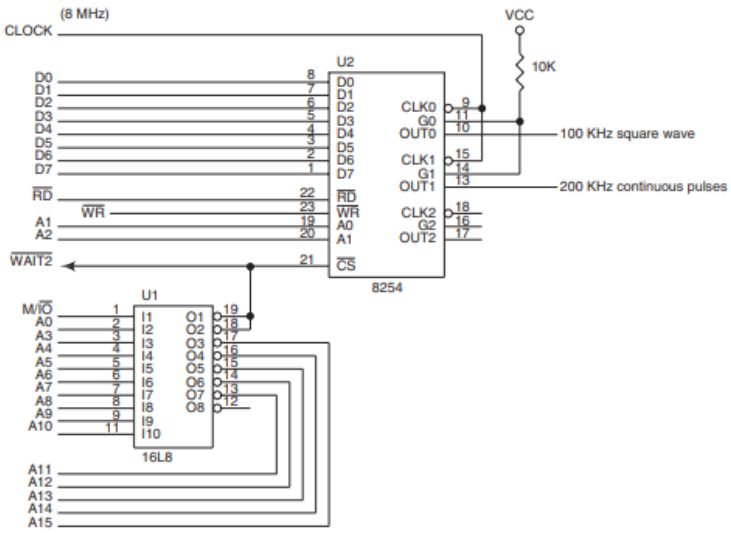
A_1	A_0	*Function*
0	0	Counter 0
0	1	Counter 1
1	0	Counter 2
1	1	Control word

8.1.13 Programming the 8254

- Control Word memungkinkan pemrogram untuk memilih penghitung, mode operasi, dan jenis operasi (baca/tulis).
- Control Word juga memilih hitungan biner atau BCD.
- Setiap pencacah dapat diprogram dengan hitungan 1 hingga FFFFH.
- Hitungan 0 sama dengan FFFFH+l (65.536) atau 10.000 dalam BCD.
- Hitungan minimum 1 berlaku untuk semua mode operasi kecuali mode 2 dan 3, yang memiliki hitungan minimum 2.



Generating a Waveform with the 8254

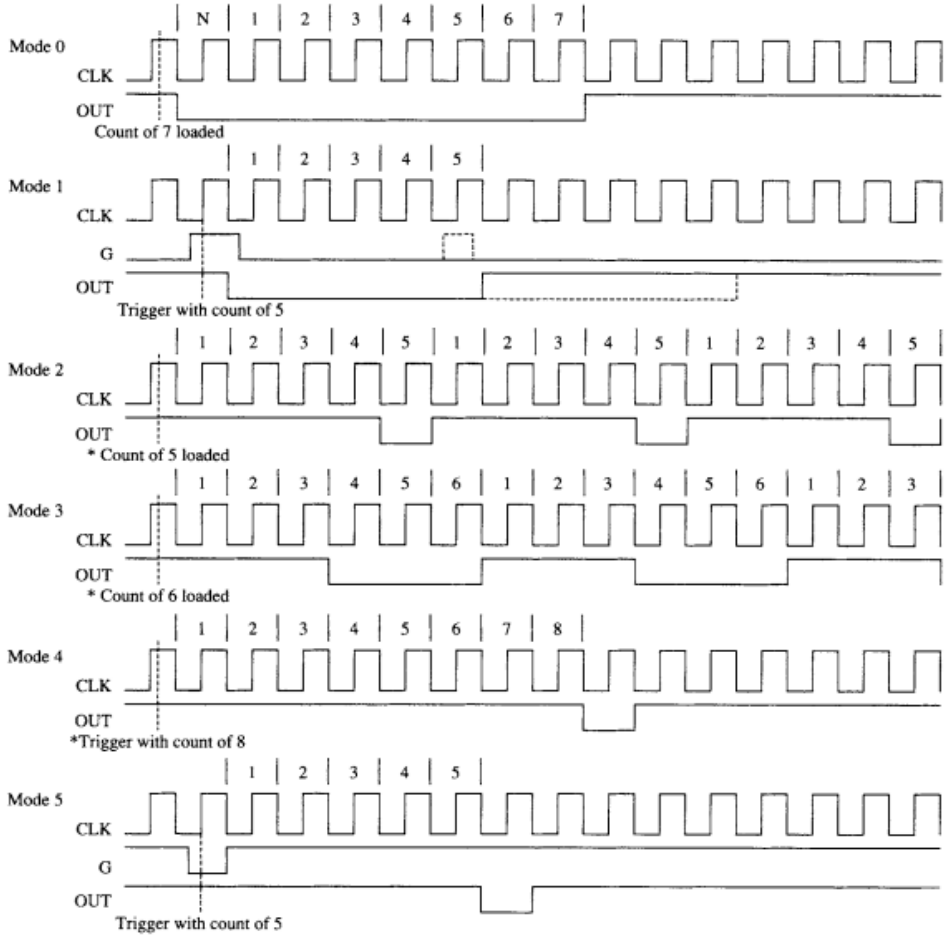


- Gambar di atas menunjukkan 8254 terhubung ke fungsi pada port I/O 0700H, 0702H, 0704H, dan 0706H dari mikroprosesor 80386SX. Alamat

diterjemahkan dengan menggunakan PLD yang juga menghasilkan sinyal write strobe untuk 8254, yang terhubung ke koneksi bus data orde rendah.

- PLD menghasilkan sinyal tunggu untuk mikroprosesor yang menyebabkan dua status tunggu saat 8254 diakses.

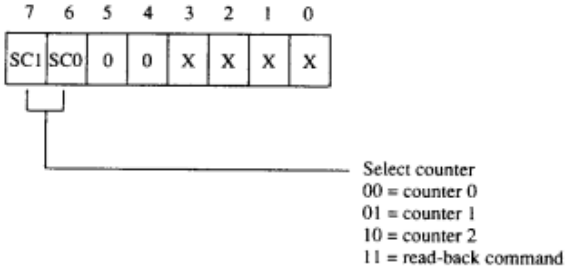
Mode of Operation



8.1.14 Reading a Counter

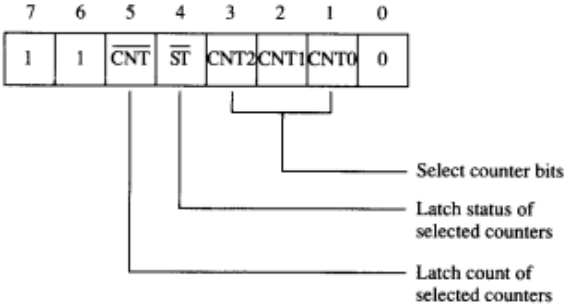
- Setiap penghitung memiliki latches internal yang dibaca dengan read counter port. Latch ini biasanya akan mengikuti hitungan.
- Jika isi counter diperlukan, maka latch dapat mengingat hitungan dengan memprogram counter latch control word (lihat Gambar 11–37), yang menyebabkan isi pencacah ditahan dalam latch hingga terbaca.
- Setiap kali pembacaan dari latch atau penghitung diprogram, latch melacak isi penghitung.

FIGURE 11–37 The 8254-2 counter latch control word.

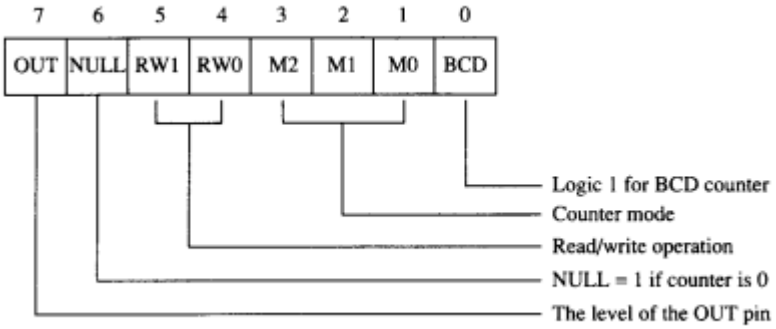


- Untuk lebih dari satu pencacah untuk dibaca pada saat yang sama, digunakan read-back control word, diilustrasikan pada Gambar 11–38. Dengan read-back control word, bit CNT adalah logika 0 untuk menyebabkan penghitung yang dipilih oleh CNT0, CNT1, dan CNT2 terkunci. Jika register status akan dikunci, maka bit ST ditempatkan pada logika 0.

FIGURE 11–38 The 8254-2 read-back control word.



Gambar 11–39 menunjukkan register status, yang menunjukkan status pin output, apakah pencacah berada pada keadaan nol (0), dan bagaimana pencacah diprogram.

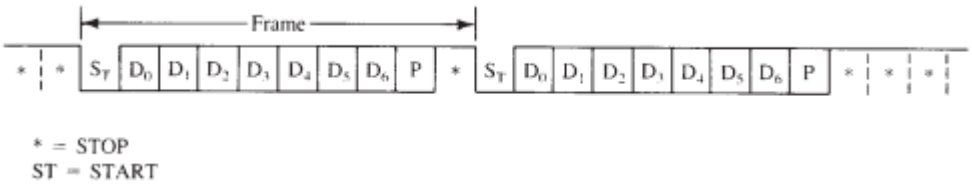


16550 Programmable Communications Interface

- 16550 = penerima/pemancar asyn chronous universal (UART) yang sepenuhnya kompatibel dengan mikroprosesor Intel.
- 16550 mampu beroperasi pada baud 0–1,5 M. Baud rate adalah jumlah bit yang ditransfer per detik (bps), termasuk mulai, berhenti, data, dan paritas (Bps adalah byte per detik dan bps adalah bit per detik).

Asyncrhronous Serial Data

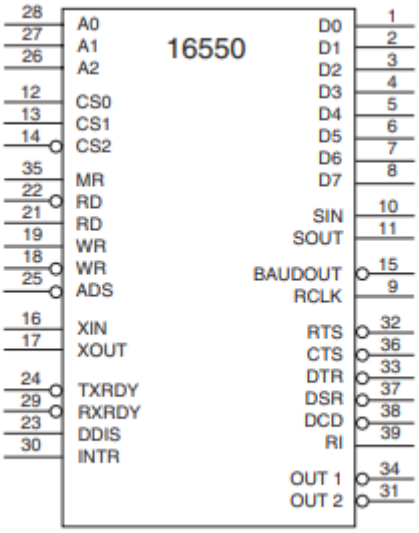
- Data serial asinkron ditransmisikan dan diterima tanpa jam atau sinyal waktu. Gambar dibawah ini mengilustrasikan dua bingkai data serial asinkron.
- Setiap frame berisi bit awal, tujuh bit data, paritas, dan satu stop bit. Angka tersebut menunjukkan bingkai yang berisi satu karakter ASCII dan 10 bit.



16550 Functional Description

- Dua bagian yang sepenuhnya terpisah bertanggung jawab untuk komunikasi data: penerima dan pemancar. Karena masing-masing bagian ini independen,
- 16550 dapat berfungsi dalam mode implex, half duplex, atau full-duplex. Salah satu fitur utama 16550 adalah penerima internal dan memori FIFO (masuk pertama, keluar pertama).
- Karena masing-masing sedalam 16 byte, UART hanya memerlukan perhatian dari mikroprosesor setelah menerima 16 byte data. Itu juga menampung 16 byte sebelum mikroprosesor harus menunggu pemancar.
- FIFO menjadikan UART ini ideal saat berinteraksi dengan sistem berkecepatan tinggi karena lebih sedikit waktu yang diperlukan untuk menyervisnya.

- 16550 dapat mengontrol modem (modulator/demodulator), yang merupakan perangkat yang mengubah level TTL dari data serial menjadi nada audio yang melewati sistem telepon.



Programming the 16550

- Pemrograman adalah proses dua bagian yang mencakup dialog inisialisasi dan dialog operasional.
- Di PC, yang menggunakan 16550 atau program yang setara, alamat port I/O didekodekan pada 3F8H hingga 3FFH untuk port COM 0 dan 2F8H hingga 2FFH untuk port COM 2.

Initializing the 16550

- Dialog inisialisasi, yang terjadi setelah reset perangkat keras atau perangkat lunak, terdiri dari dua bagian: memprogram line control register dan baud rate generator.
- Line control register memilih jumlah bit data, jumlah bit stop, dan paritas (apakah genap atau ganjil, atau jika paritas dikirim sebagai 1 atau 0).
- Generator baud rate diprogram dengan pembagi yang menentukan baud rate bagian pemancar.

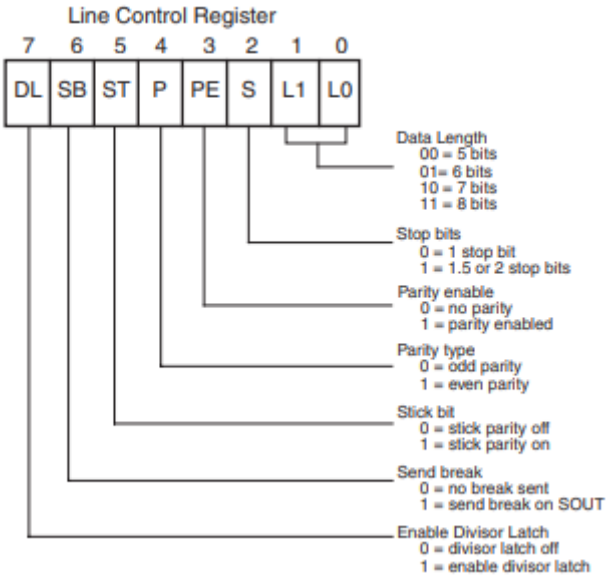


TABLE 11–6 The operation of the ST and parity bits.

ST	*P*	*PE*	*Function*
0	0	0	No parity
0	0	1	Odd parity
0	1	0	No parity
0	1	1	Even parity
1	0	0	Undefined
1	0	1	Send/receive 1
1	1	0	Undefined
1	1	1	Send/receive 0

Programming the Baud Rate

- Generator baud rate diprogram pada alamat I/O 000 dan 001 (A2, A1, A0).
- Port 000 digunakan untuk menampung bagian yang paling tidak signifikan dari pembagi 16-bit dan port 001 digunakan untuk menampung bagian yang paling signifikan.
- Nilai yang digunakan untuk pembagi bergantung pada jam eksternal atau frekuensi kristal. Tabel 11–7 mengilustrasikan baud rate umum diperoleh jika kristal 18,432 MHz digunakan sebagai sumber waktu.

TABLE 11–7 The divisor used with the baud rate generator for an 18.432 MHz crystal illustrating common baud rates.

Baud Rate	*Divisor Value*
110	10,473
300	3840
1200	920
2400	480
4800	240
9600	120
19,200	60
38,400	30
57,600	20
115,200	10

Sending Serial Data

- Sebelum data serial dapat dikirim atau diterima melalui 16550, perlu diketahui fungsi register status baris (lihat Gambar 11–47).
- Line status register berisi informasi tentang kondisi kesalahan dan keadaan pemancar dan penerima. Register ini diuji sebelum sebuah byte ditransmisikan atau dapat diterima.

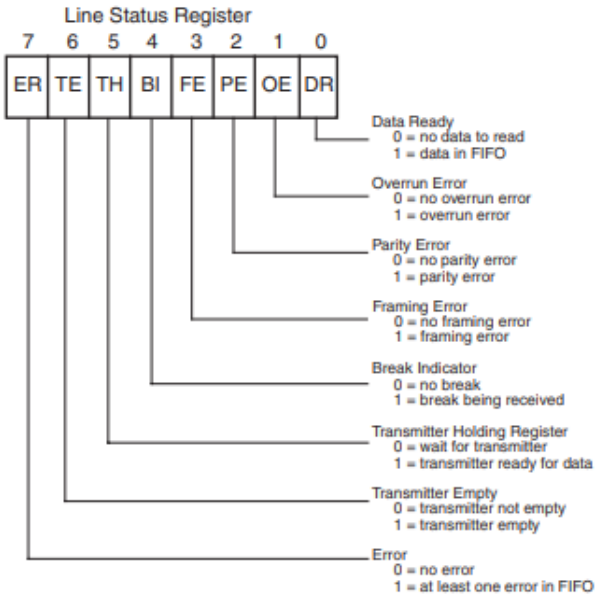


FIGURE 11–47 The contents of the line status register of the 16550 UART.

Receiving Serial Data

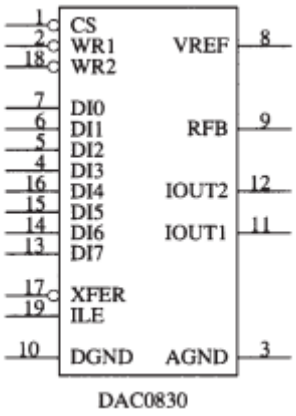
- Untuk membaca informasi yang diterima dari 16550, uji bit DR dari register status baris.

UART Errors

- Jenis error yang terdeteksi oleh 16550 adalah parity error, framing error, dan over run error.
- Parity error menunjukkan bahwa data yang diterima berisi paritas yang salah.
- Framing error menunjukkan bahwa bit mulai dan berhenti tidak berada di tempat yang semestinya.
- Over run error menunjukkan bahwa data telah melebihi buffer FIFO penerima internal.

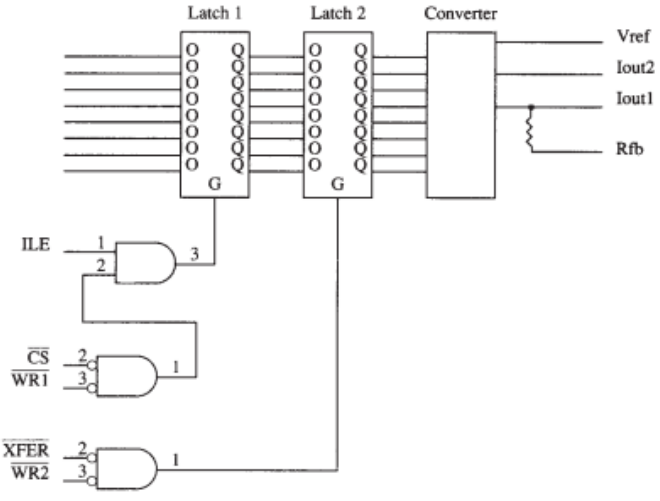
8.1.15 Analog-To-Digital (ADC) and Digital-To-Analog (DAC) Converters

The DAC0830 Digital-to-Analog Converter



- Jumlah langkah tegangan yang dihasilkan oleh konverter sama dengan jumlah kombinasi input biner.
- Konverter 8-bit menghasilkan 256 level tegangan yang berbeda, konverter 10-bit menghasilkan 1024 level, dan seterusnya.
- DAC0830 adalah konverter kecepatan sedang yang mengubah input digital menjadi output analog dalam waktu sekitar 1,0 μs.

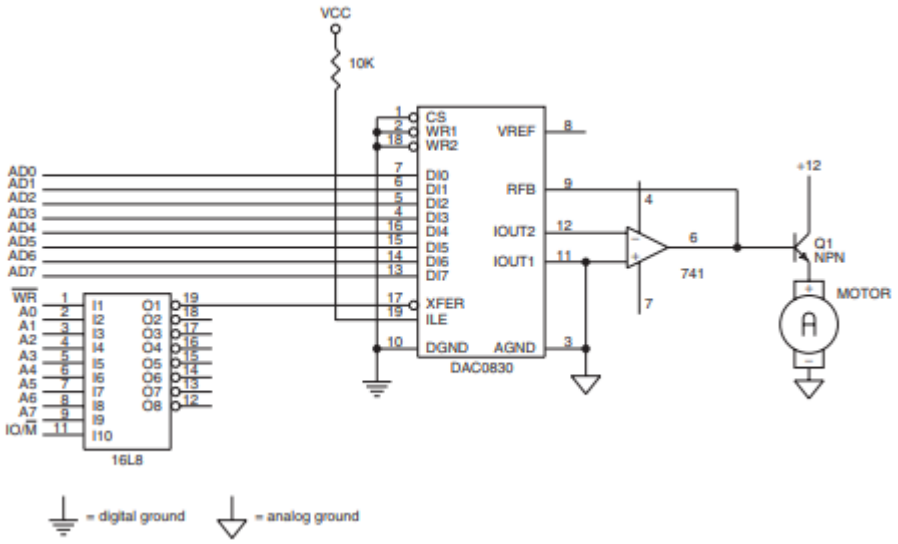
Internal Structure of The DAC0830



Connecting the DAC0830 To The Micrprosessor

- DAC0830 terhubung ke prosesor mikro seperti yang diilustrasikan pada Gambar 11–50. Di sini, PLD digunakan untuk mendekode DAC0830 pada I/O 8-bit alamat port 20H.

- Setiap kali instruksi OUT 20H,AL dijalankan, isi koneksi bus data AD0–AD7 diteruskan ke konverter di dalam DAC0830.
- Penguat operasional 741, bersama dengan tegangan referensi zener -12 V, menyebabkan tegangan output skala penuh sama dengan +12 V.
- Output dari penguat operasional memberi daya kepada driver yang menggerakkan motor 12 V DC.



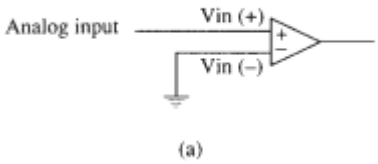
The ADC080X Analog-to-Digital Converter

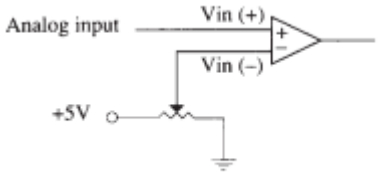
- ADC080X membutuhkan hingga 100 μs untuk mengubah tegangan masukan analog menjadi kode keluaran digital.

The Analog Input Signal

- Ada dua input analog ke ADC0804: VIN(+) dan VIN(-). Input ini terhubung ke penguat operasional internal dan merupakan input diferensial, seperti yang ditunjukkan pada Gambar 11–53.
- Input diferensial dijumlahkan oleh penguat operasional untuk menghasilkan sinyal untuk konverter analog-ke-digital internal.

FIGURE 11–53 The analog inputs to the ADC0804 converter. (a) To sense a 0- to +5.0 V input. (b) To sense an input offset from ground.



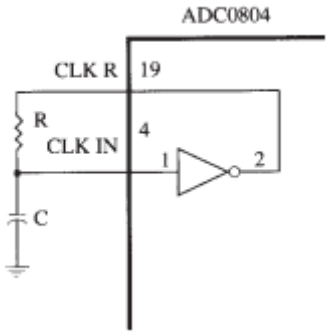


(b)

Generating The Clock Signal

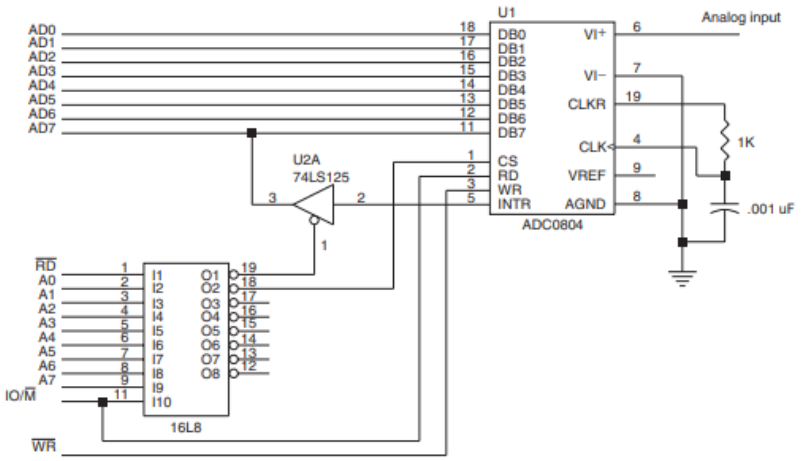
- ADC0804 memerlukan sumber clock untuk pengoperasian.
- Clock dapat berupa clock eksternal yang diterapkan pada pin CLK IN atau dapat dihasilkan dengan rangkaian RC.
- Rentang frekuensi clock yang diizinkan adalah antara 100 KHz dan 1460 KHz.
- Jika clock dihasilkan dengan rangkaian RC, maka digunakan pin CLK IN dan CLK R yang terhubung ke rangkaian RC, seperti yang diilustrasikan pada Gambar 11–54. Saat koneksi ini sedang digunakan, jam frekuensi dihitung dengan persamaan berikut:

$$Fclk = \frac{1}{1.1RC}$$



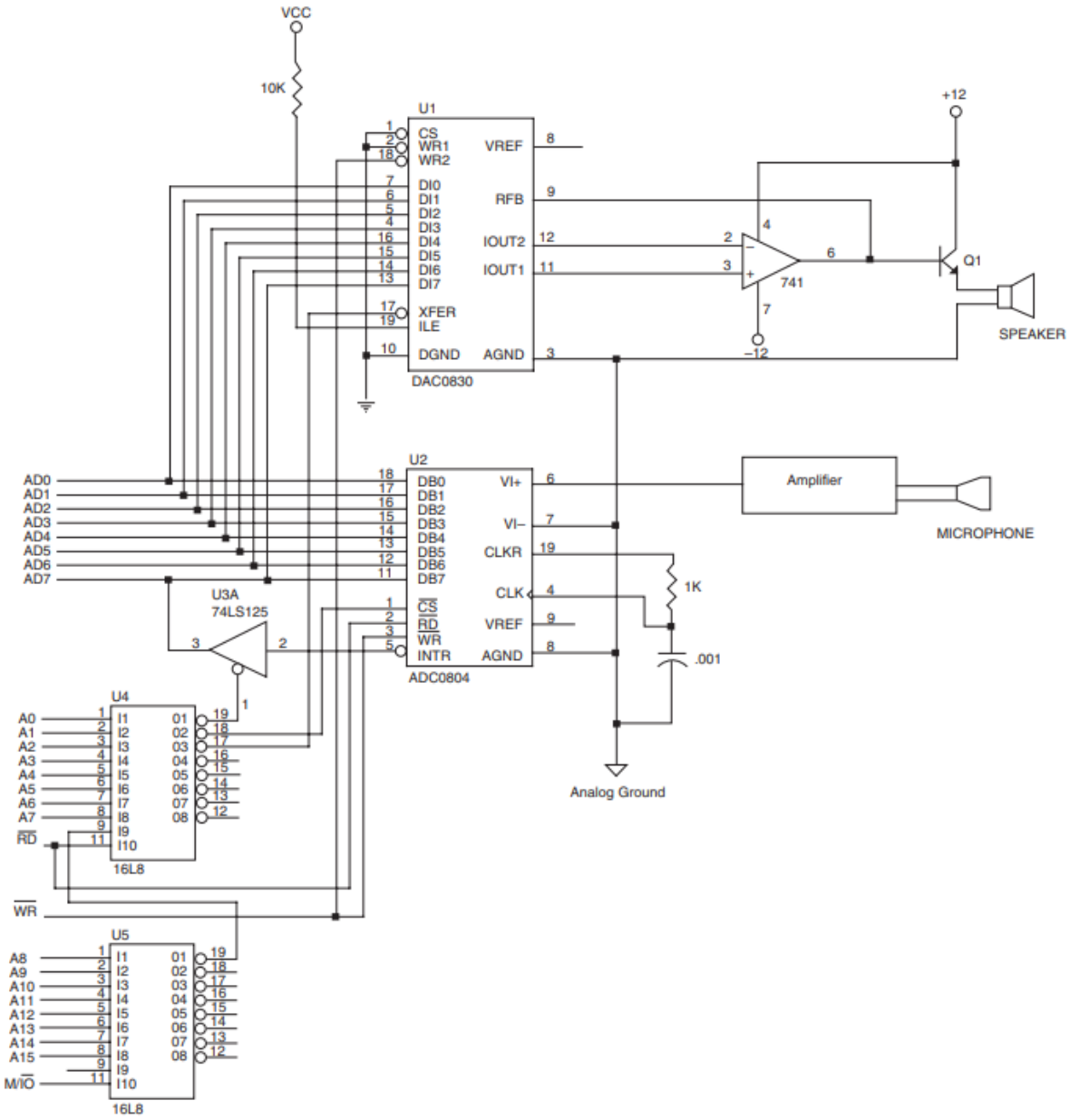
Connecting the ADC0804 To The Microprosessor

- ADC0804 dihubungkan ke prosesor mikro 8086, seperti yang diilustrasikan pada gambar dibawah.
- Misalkan ADC0804 didekodekan pada alamat port I/O 8-bit 40H untuk data dan alamat port 42H untuk sinyal INTR, dan prosedur diperlukan untuk memulai dan membaca data dari ADC.



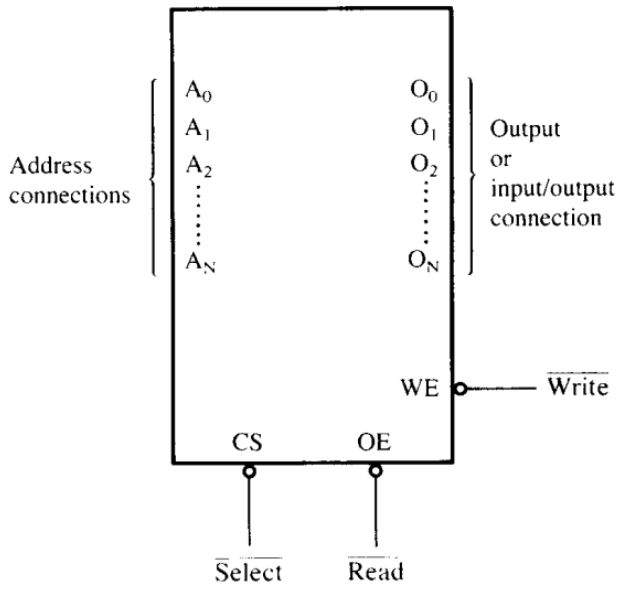
Using the ADC0804 and The DAC0830

- ADC0804 dan DAC0830 digunakan untuk menangkap dan memutar ulang sinyal audio atau ucapan dengan kualitas manusia.
- ADC0804 digunakan untuk menangkap sinyal audio dan menyimpannya dalam memori untuk pemutaran nanti melalui DAC0830.
- Port I/O 0700H dan 0702H digunakan untuk menghubungkan ADC0804, sementara DAC0830 dihubungkan pada port I/O 704H.
- Speech diambil sampelnya dan disimpan di bagian memori yang disebut WORDS.
- Tingkat sampel dipilih pada 2048 sampel per detik, yang menghasilkan ucapan yang terdengar dapat diterima.



8.2 Interfacing Memory

- 4 tipe umum atau jenis-jenis memory:
  1) ROM (Read Only Memory)
  2) EEPROM (Flash Memory)
  3) SRAM (Static Random Access Memory)
  4) DRAM (Dynamic Random Access Memory)
- Berikut ini adalah ilustrasi pseudo-memory component



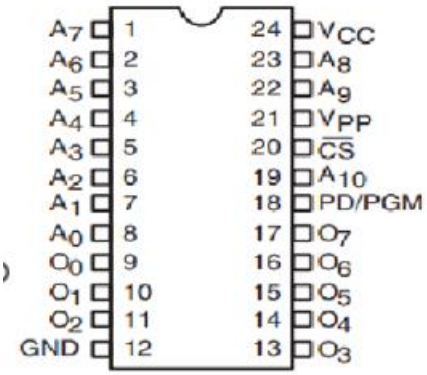
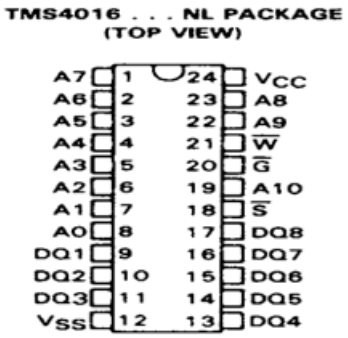
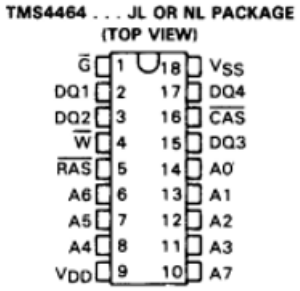
→Address connection connect dengan address bus, sementara output atau input/output connection terhubung dengan data bus.

- Selection Connection : tiap memory memiliki input yang akan menyeleksi memory device. Selection connection terdiri dari tiga nama dengan fungsi yang sama, yaitu:
  1) Chip Select ($\overline{CS}$)
  2) Chip Enable ($\overline{CE}$)
  3) Select ($\bar{S}$)

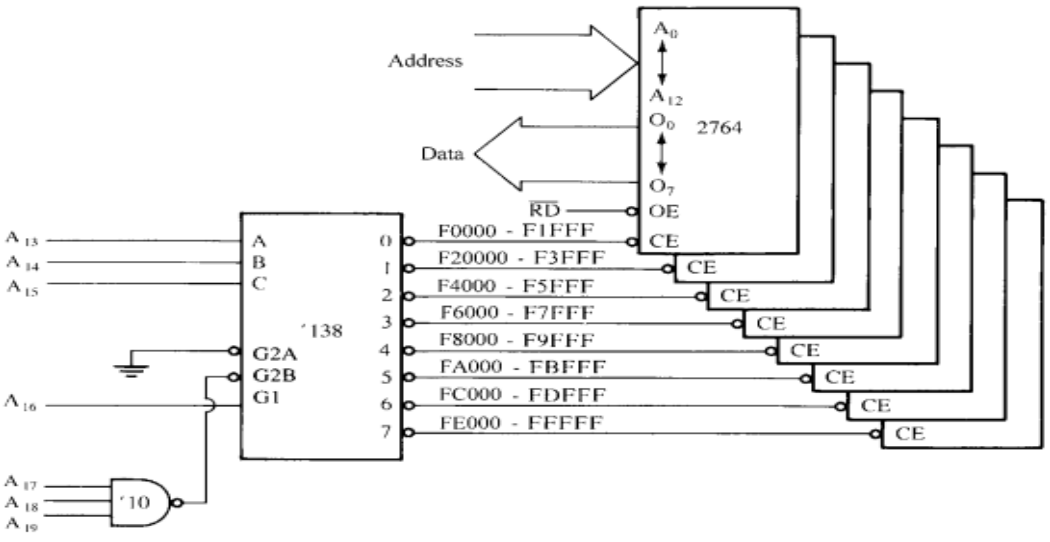
  Semua nama selection connection merupakan active low yang hanya akan aktif atau menjalankan operasi baca atau tulis ketika diberi logic 0.

  RAM minimal memiliki 1 selection $\overline{CS}$ atau $\bar{S}$. Sedangkan ROM minimal memiliki 1 selection $\overline{CE}$.

- Control Connection : semua memory devices memiliki bentuk control input atau output. ROM memiliki 1 control input sedangkan RAM memiliki 2 control inputs.
  Control Input ROM : $\overline{OE}$ (output enable) atau $\bar{G}$ (gate). Jika $\overline{OE}$ dan $\overline{CE}$ aktif, maka outputnya akan enable, jika $\overline{OE}$ tidak aktif maka output disable.
  Control Input RAM : Jika hanya ada control input R/$\overline{W}$ maka pin tersebut dapat memilih operasi baca atau tulis (read/write) namun hanya jika selectionnya adalah $\overline{CS}$. Jika ada dua control inputs maka control inputnya adalah $\overline{WE}$ (write enable) dan $\overline{OE}$.
- Berikut ini adalah proses untuk mengetahui jumlah saluran alamat dan cara membaca tipe-tipe ROM, SRAM, EEPROM, dan DRAM.

EEPROM (27xx)	SRAM (40xx, 62xx)
 The pin-out of the 2716, 2K × 8 EPROM. (Courtesy o 2716 → 16/8 = 2 → 2K $2^{11} = 2048 (\approx 2K)$ Kapasitas : 8x2Kbyte Saluran Alamat : 11	 4016 → 16/8 = 2 → 2K $2^{11} = 2048 (\approx 2K)$ Kapasitas : 8x2Kbyte Saluran Alamat : 11
ROM (27xx)	**DRAM (44xx, 41xx)**
Jika tipenya 2708 2708 → 8/8 = 1 → 1K $2^{10} = 1024 (\approx 1K)$ Kapasitas : 8x1Kbyte Saluran Alamat : 10 Jika tipenya 2704 2704 → 4/8 = 1/2 → 512 $2^{9} = 512$ Kapasitas : 8x512byte Saluran Alamat : 9	 4464 → 64x4 Kapasitas : 4x64Kbyte

- 3 to 8 Line Decoder 74LS138



Berikut ini cara mendapatkan alamat awal dan alamat akhir tiap pin:

- Kapasitas memory : 8x8Kbyte.
- Saluran Alamat : 13 (A12 – A0).
- Tipe : 2764 → 64Kbyte = 8 IC memory dan 8Kbyte kapasitas.
- Alamat awal = 0 | 0000 | 0000 | 0000 | = 00000

  Alamat akhir = 1 | 1111 | 1111 | 1111 | = 01FFF
- Awal = F0000H + 01FFF = F1FFF
- Memory Map

F0000H	F1FFFH	8Kbyte
F2000H	F3FFFH	8Kbyte
F4000H	F5FFFH	8Kbyte
F6000H	F7FFFH	8Kbyte
F8000H	F9FFFH	8Kbyte
FA000H	FBFFFH	8Kbyte
FC000H	FDFFFH	8Kbyte
FE000H	FFFFFH	8Kbyte

8.3 LATIHAN

1. Jelaskan cara kerja rangkaian berikut

346 CHAPTER 10

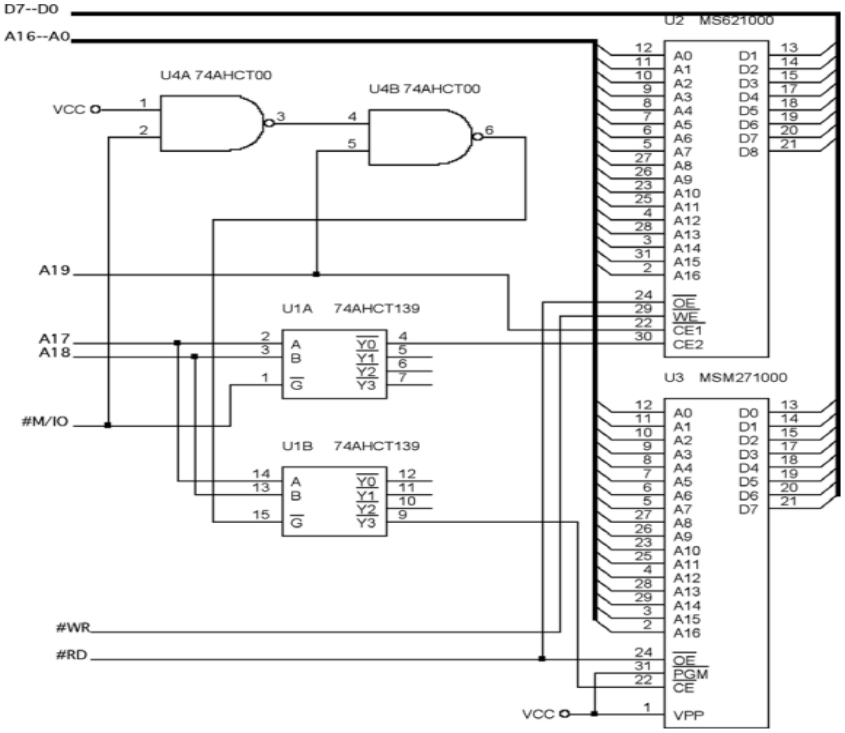


FIGURE 10–17 A sample memory system constructed with a 74HCT139.

- Pada rangkaian ini terdapat 4 saluran yang menjadi perhatian utama, yaitu A19, A17, A18, serta #M/IO. Saluran #M/IO merupakan control input ROM yang akan mengaktifkan output ketika gate dan chip enablenya aktif. #M/IO terkoneksi dengan pin $\bar{G}$ yang bersifat active low, sehingga perlu diberi logic 0 untuk mengaktifkan pin $\bar{G}$ pada U1A. Saluran A17 dan A18 terkoneksi dengan pin A dan B pada U1A. Pin A dan B bersifat active high dan outputnya terhubung ke CE2. Untuk mengaktifkan CE2, maka pin A dan B melalui saluran A17 dan A18 perlu diberi logic 1 pada masing-masing salurannya. Dengan begitu, output U1A yang terhubung ke CE2 menjadi aktif. Saluran A19 terhubung ke gerbang NAND dan $\overline{CE1}$, sehingga A19 perlu diberi logic 0 karena $\overline{CE1}$ bersifat active low dan output dari NAND gate (kaki 6) akan menghasilkan logic 1 sehingga pin $\bar{G}$ pada U1B akan non aktif. Ketika U1B non aktif, maka hanya memory U2 yang dapat diakses. Untuk mengakses U3, maka saluran #M/IO perlu diberi logic 0, A17 dan A18 diberi logic 1, sedangkan A19 perlu diberi logic 1 sehingga output NAND gate akan mengaktifkan pin $\bar{G}$ U1B.

2. Jelaskan cara kerja rangkaian berikut

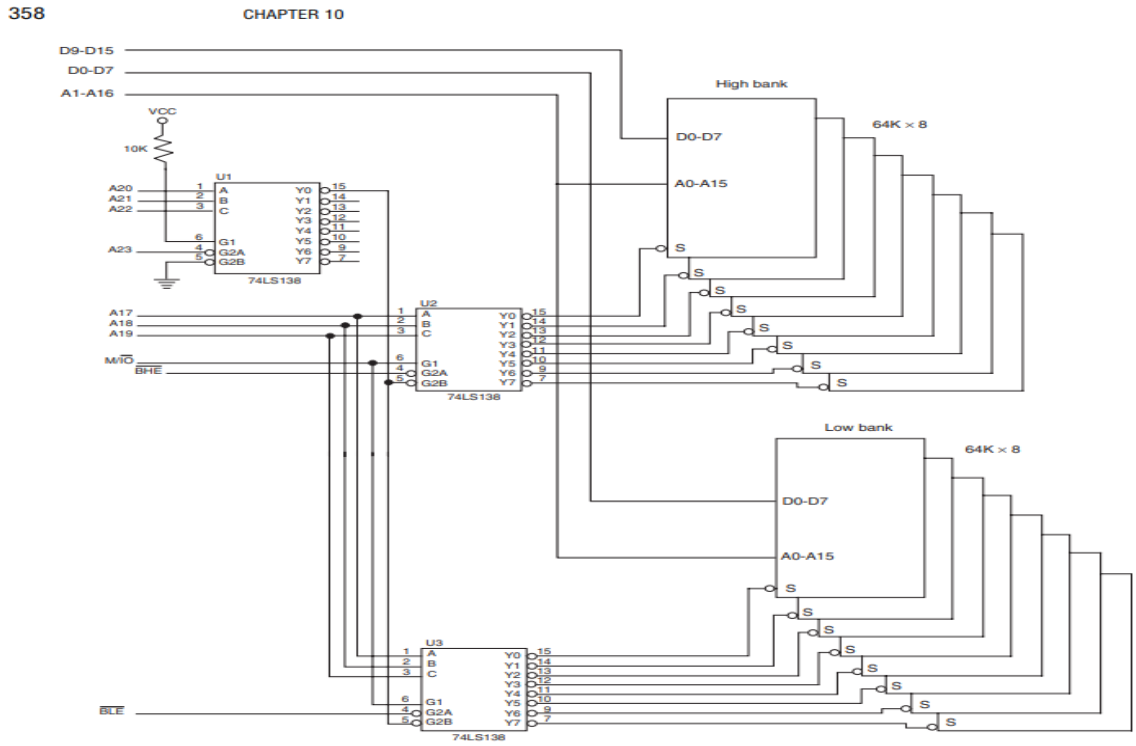


FIGURE 10–28 Separate bank decoders.

- Pada rangkaian ini terdapat dua saluran utama, yaitu $\overline{BHE}$ dan $\overline{BLE}$. Ketika$\overline{BLE}$ diberi logic 0 maka memory pada Low Bank akan bisa diakses. Sedangkan ketika $\overline{BHE}$ diberi logic 0 maka memory pada High Bank akan diakses. Jika ingin mengaktifkan semua gate, maka digunakan saluran M/IO. Ketika M/IO diberi logic 1, maka pin G1 pada U2 dan U3 akan aktif, sedangkan pin G2B dapat diaktifkan melalui saluran A20, A21, dan A22 dengan memberi logic 0 sehingga pin output U1 akan menghasilkan 0 dan G2B akan aktif karena sifatnya yang active low. $\overline{BHE}$ dan $\overline{BLE}$ diberi logic 0 atau 1 sesuai dengan keinginan akses. Jika ingin mengakses High Bank maka $\overline{BHE}$ diberi logic 0 dan $\overline{BLE}$ diberi logic 1. Begitupun sebaliknya.

Tugas 1: Mikroprosesor dengan 20 saluran alamat (A19-A0). EPROM 1 kapasitas 2Kbyte, EPROM 2 kapasitas 4Kbyte, RAM 1 kapasitas 2Kbyte, dan RAM 2 kapasitas 4Kbyte.

- **Memory Map**

  20 SA → 5 digit hexadecimal.

  20 SA = 1Mbyte = 1024 Kbyte

$00000 $007FF	EPROM 1 2Kbyte
$00800 $017FF	EPROM 2 4Kbyte
$01800 $01FFF	RAM 1 2Kbyte
$02000 $02FFF	RAM 2 4KByte
$03000 $FFFFF	Kosong 1012KByte

  Nilai-nilai memory map tersebut didapatkan melalui perhitungan masing-masing saluran alamat seperti berikut:

  EPROM 1: 11 SA dari 20 SA

  Awal : 0|0|0000|0000|0000 = 00000

  Akhir: 0|0|0111|1111|1111 = 007FF

  EPROM 2: 12 SA dari 20 SA

  Awal : 00800

  12 SA: 00FFF

  Awal+SA = 017FF

  RAM 1: 11 SA dari 20 SA

  Awal : 01800

  11 SA: 007FF

  Awal+SA = 01FFF

  RAM 2: 12 SA dari 20 SA

  Awal : 02000

  12 SA: 00FFF

  Awal+SA = 02FFF

- **Conventional Method** (Tulis alamat awal-akhir dari tiap chip)

	A19	A18	A17	A16	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Ket
EP1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A0-A10 (CS0)
	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	
EP2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	A0-A12 (CS1)
	0	0	0	0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	
R1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	A0-A12 (CS2)
	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	
R2	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	A0-A13 (CS3)
	0	0	0	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	

CS0 = common address → tidak terhubung ke rangkaian interfacing.

- **Perhitungan jumlah dan nomor kavling**

Keterangan	**KByte**	**Jumlah Kavling**	**No Kavling**
CS0	2	1	0
CS1	4	2	1,2
CS2	2	1	3
CS3	4	2	4,5
Total		**6 kavling**	**3 bit**

- **Tabel I/O Decoder**

INPUT			OUTPUT			
A13	A12	A11	CS3	CS2	CS1	CS0
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	1	0	1
0	1	1	1	0	1	1
1	0	0	0	1	1	1
1	0	1	0	1	1	1
1	1	0	x	x	x	x
1	1	1	x	x	x	x

- **K-MAPS**

CS0 = A13 + A12 + A11

A13\A12A11	00	01	11	10
0	0	1	1	1
1	1	1	d	d

CS1 = A13 + $\overline{A12}$ $\overline{A11}$ + A12 A11

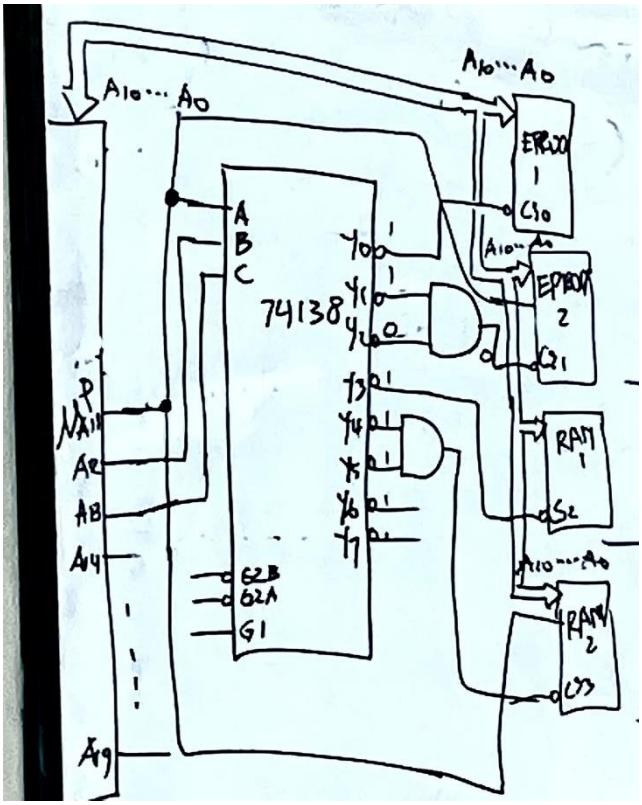
A13\A12A11	00	01	11	10
0	1	0	1	0
1	1	1	d	d

CS2 = $\overline{A12} + \overline{A11}$

A13\A12A11	00	01	11	10
0	1	1	0	1
1	1	1	d	d

CS3 = $\overline{A13}$

A13\A12A11	00	01	11	10
0	1	1	1	1
1	0	0	d	d



DAFTAR PUSTAKA

Abdul-Kader, S. A. A. (2009). Emulation of the Microprocessor Intel 80386. *Diyala Journal of Engineering Sciences*, *2*(1), 13-34.

Ayala, K. J. (1995). 8086 microprocessor: programming & interfacing the PC.

Brey, B. B. (1999). *The Intel microprocessors 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium II processors: architecture, programming, and interfacing*. Prentice-Hall, Inc..

Dauglas V. Hall. (1986). Microprocessor and Interfacing Programing and Hardware. New York: Mc Graw Hill

Hyde, R. (2003). *The art of assembly language* (Vol. 75). San Francisco, CA, USA: No Starch Press.

Jacob, J. A., & Chow, P. (1999, February). Memory interfacing and instruction specification for reconfigurable processors. In *Proceedings of the 1999 ACM/SIGDA seventh international symposium on Field programmable gate arrays* (pp. 145-154).

Knaggs, P., & Welsh, S. (2004). *ARM: Assembly Language Programming*. Bournemouth University, School of Design, Engineering, and Computing.

Krishna, D. C., Sridevi, U., & Anjaiah, B. (2016). An Effective Assembly Language Routine of 8051 Microcontroller to Convert Hexadecimal Number to Decimal Number and Displaying the Decimal Digits at 8255 PPI Ports. *International Journal of Engineering Science*, *3523*.

Triebel, W. A., & Singh, A. (1991). *The 8088 and 8086 microprocessors: programming, interfacing, software, hardware, and applications*. Prentice-Hall, Inc..

Tutorialspoint. (2023). Microprocessor – 8086 Overview. [Tutorialspoint.com]. Diakses pada https://www.tutorialspoint.com/microprocessor/microprocessor_8086_overview.html

Wardoyo, S. (2011). Dasar Mikroprosesor.